

Introduction to Data Science and Systems

Lecture 2: Computational Linear Algebra

Dr Iraklis Klampanos

University of Glasgow

Recap of the last lecture

Vectors

- **scalar multiplication** so that $a\mathbf{x}$ is defined for any scalar a. For real vectors, $a\mathbf{x} = [ax_1, ax_2, \ldots ax_n]$, elementwise scaling.

$$(\mathbb{R}, \mathbb{R}^n) \rightarrow \mathbb{R}^n$$

- **vector addition** so that $\mathbf{x} + \mathbf{y}$ vectors $\mathbf{x}, \mathbf{y}$ of equal dimension. For real vectors, $\mathbf{x} + \mathbf{y} = [x_1 + y_1, x_2 + y_2, \ldots x_d + y_d]$ the elementwise sum

$$(\mathbb{R}^n, \mathbb{R}^n) \rightarrow \mathbb{R}^n$$

- A **norm** $\| \mathbf{x} \|$ which allows the length of vectors to be measured.

$$\mathbb{R}_n \rightarrow \mathbb{R}_{\geq 0}$$

- An **inner product** $\langle \mathbf{x} \mid \mathbf{y} \rangle$, $\langle \mathbf{x}, \mathbf{y} \rangle$ or $\mathbf{x} \cdot \mathbf{y}$ which allows the angles of two vectors to be compared. The inner product of two orthogonal vectors is 0 . For real vectors $\mathbf{x} \cdot \mathbf{y} = x_1 y_1$

$$x_3 y_3 \ldots x_d y_d$$

$$(\mathbb{R}^n, \mathbb{R}^n) \rightarrow \mathbb{R}$$

University of Glasgow

Operations with matrices

- They can be **added** and **subtracted** $C = A + B$

$$(\mathbb{R}^{n\times m}, \mathbb{R}^{n\times m}) \to \mathbb{R}^{n\times m}$$

- They can be **scaled** with a scalar $C = sA$

$$(\mathbb{R}^{n\times m}, \mathbb{R}) \to \mathbb{R}^{n\times m}$$

- They can be **transposed** $B = A^T$; this exchanges rows and columns

$$\mathbb{R}^{n\times m} \to \mathbb{R}^{m\times n}$$

- They can be **applied** to vectors $\mathbf{y} = A\mathbf{x}$; this applies a matrix to a vector. (linear maps)

$$(\mathbb{R}^{n\times m}, \mathbb{R}^{m}) \to \mathbb{R}^{n}$$

- They can be **multiplied** together $C = AB$; this composes the effect of two matrices (composed maps)

Matrices and linear operators

Matrices are 2D arrays of reals that define **linear maps**;

- Vectors represent “points in space"
- Matrices represent *operations* that do things to those points in space.

The operations represented by matrices are a particular class of functions on vectors

University of Glasgow

Intended Learning Outcomes

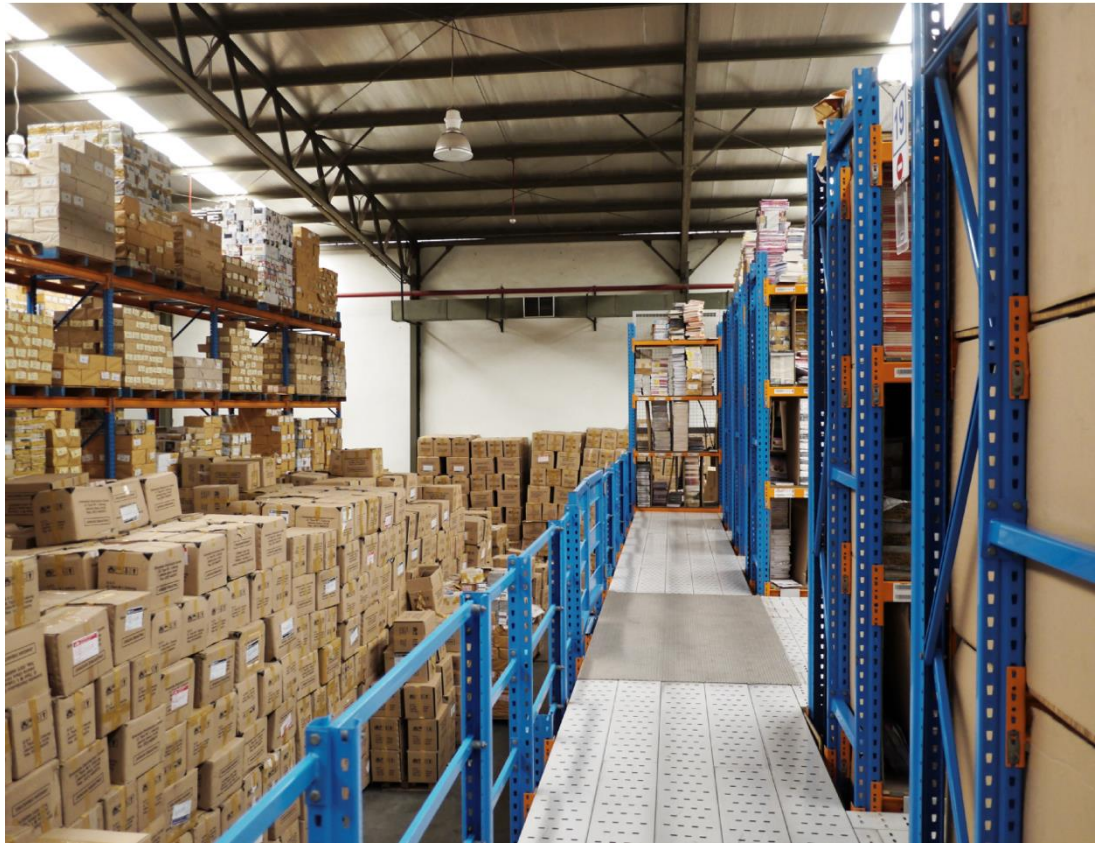
- How discrete problems can be modelled using continuous mathematics, i.e. Using matrices
  - How graphs can be represented as matrices
  - How flows on graphs can be represented as matrix operations
- What eigenvectors and eigenvalues are
  - How the power iteration method can compute them
  - How they can be used to decompose matrices
- What the trace and determinant are, and the geometric intuition underlying them
- What positive (semi-)definiteness means and why it is important
- What the singular value decomposition (SVD) is and how it can be used to compute functions of matrices
- What a linear system of equations is and how it can be represented by a matrix
  - What matrix inversion is and how it relates to solving linear systems of equations
  - The numerical problems with direct inversion
  - What the pseudo-inverse is, how it is derived from the SVD, and how it can be used
- How to normalise data by using matrix operations to "whiten" it

University of Glasgow

Graphs as matrices

- **Example**: distributing packages
- **Task:** the packages and the connectivity between distributions centres
- **Target:** to predict which warehouses are going to receive lots of packages

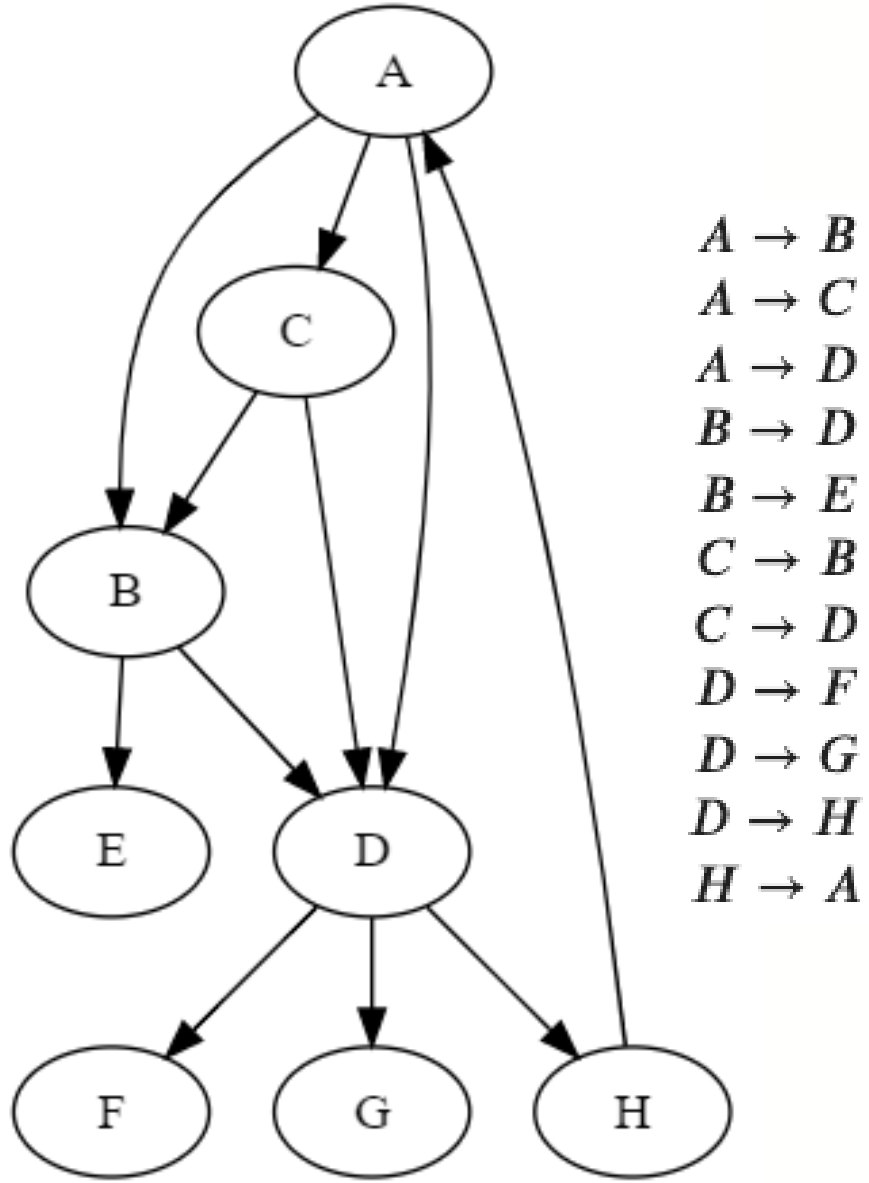
How can this problem be modelled?



Graphs as matrices

- **Linear assumption:** the flow from site to site is linear -- that is the packages arriving at one site is a weighted sum of the packages currently at each of the other sites

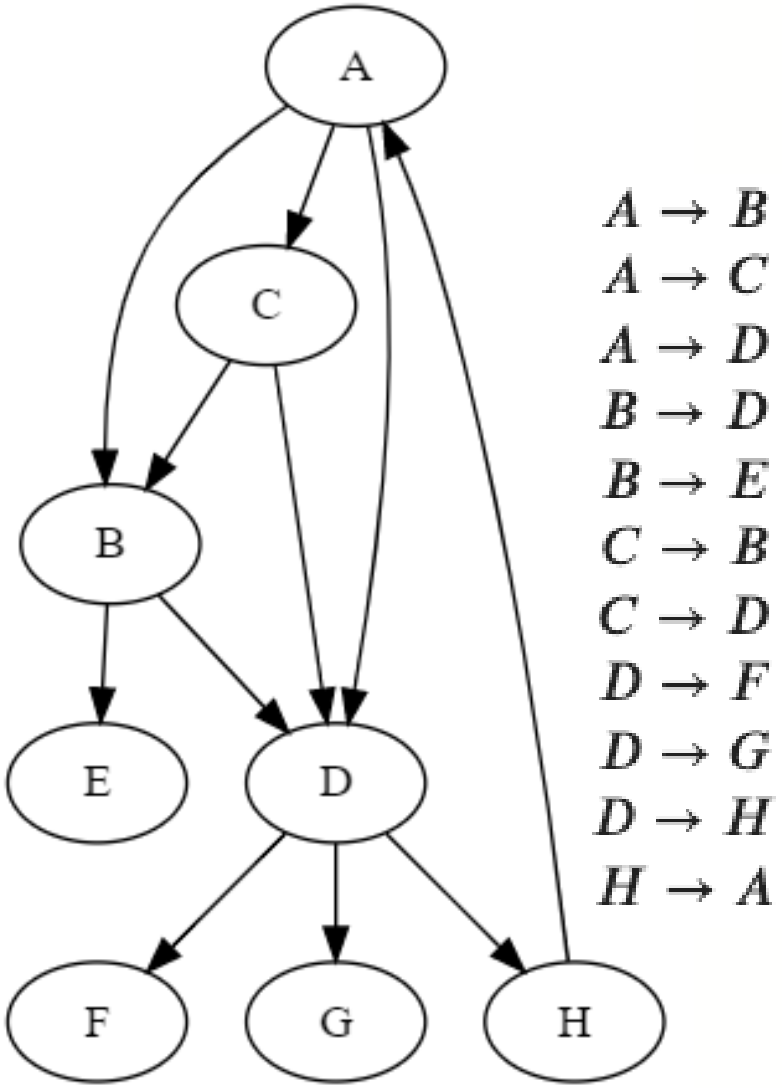
- We might model the connectivity of distribution centres as a **graph**. A **directed graph** connects **vertices** by **edges**. The definition of a graph is $G=(V,E)$, where V is a set of vertices and E is a set of edges connecting pairs of vertices.



Graphs as matrices

- We can write this as an **adjacency matrix**. We label each vertex 0,1,2,3,…
- We then create a square matrix A whose elements are all zero, except where there is an edge from V_i to V_j, in which case we set A_{ij}=1. The graph shown above has the adjacency matrix:

	A	B	C	D	E	F	G	H
A	0	1	1	1	0	0	0	0
B	0	0	0	1	1	0	0	0
C	0	1	0	1	0	0	0	0
D	0	0	0	0	0	1	1	1
E	0	0	0	0	0	0	0	0
F	0	0	0	0	0	0	0	0
G	0	0	0	0	0	0	0	0
H	1	0	0	0	0	0	0	0

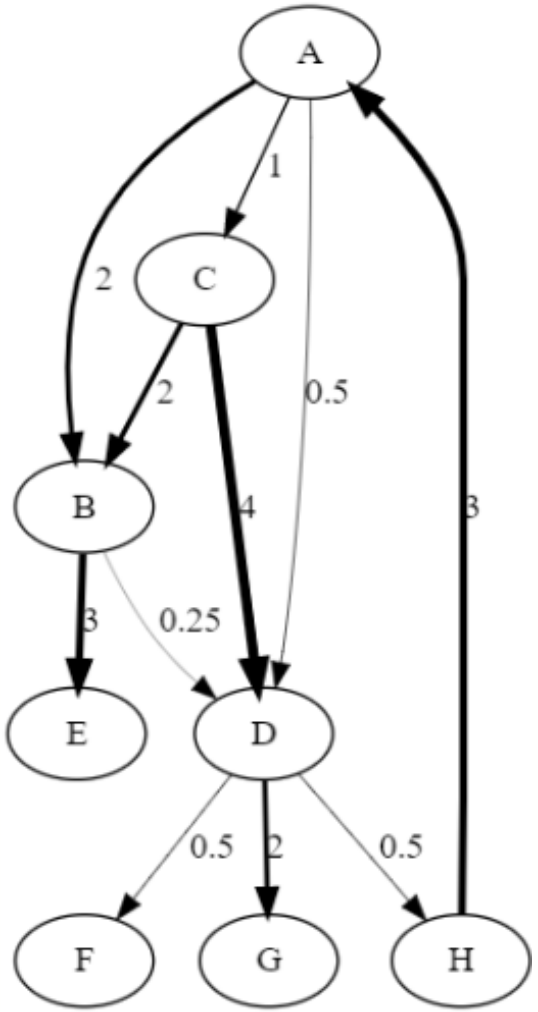


University of Glasgow

Computing graph properties

- The *out-degree* of each vertex (number of edges leaving a vertex) is the sum of each row.
- The *in-degree* of each vertex (number of edges entering a vertex) is the sum of each column.
- If the matrix is *symmetric*, it represents an undirected graph; this is the case if it is equal to its transpose.
- A directed graph can be converted to an undirected graph by computing $A'=A+A^T$. This is equivalent to making all the arrows bi-directional.
- If there are non-zero elements on the diagonal, that means there are edges connecting vertices to themselves (self-transitions).

Edge-weighted graphs



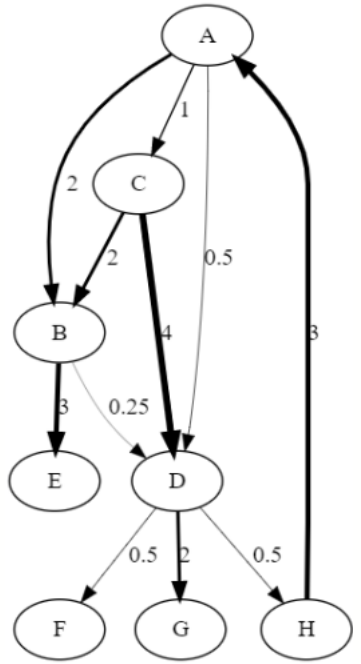
- If the some of the connections between distribution centres are stronger than others, e.g. if they are connected by bigger roads, we can model this using edge weights. Now the entry at A_{ij} represents the weight of the connection from vertex V_i to V_j.

	A	B	C	D	E	F	G	H
A	0.00	2.00	1.00	0.50	0.00	0.00	0.00	0.00
B	0.00	0.00	0.00	0.25	3.00	0.00	0.00	0.00
C	0.00	2.00	0.00	4.00	0.00	0.00	0.00	0.00
D	0.00	0.00	0.00	0.00	0.00	0.50	2.00	0.50
E	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00
F	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00
G	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00
H	3.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00

Graphs as representing flows of "mass" through a network of vertices.

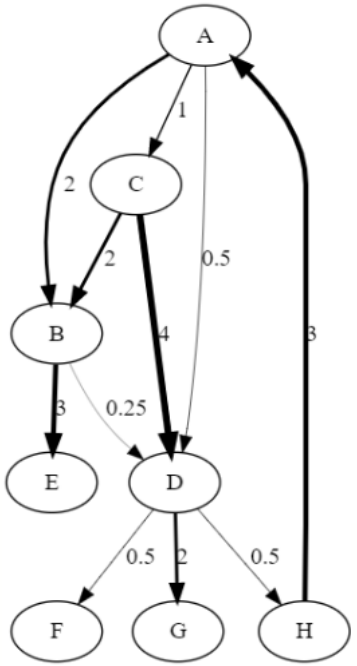
- If the total flow out of a vertex is >1, i.e. its row sums to >1, then it is a **source** of mass; for example it is *manufacturing* things.
- If the total flow out of a vertex is <1, i.e. its row sums to <1, then it is a **sink**; for example it is *consuming* things.
- If the total flow out of the vertex is 1 exactly, i.e. its row sums to 1 exactly, then it conserves mass; it only ever *re-routes* things.

	A	B	C	D	E	F	G	H
A	0.00	2.00	1.00	0.50	0.00	0.00	0.00	0.00
B	0.00	0.00	0.00	0.25	3.00	0.00	0.00	0.00
C	0.00	2.00	0.00	4.00	0.00	0.00	0.00	0.00
D	0.00	0.00	0.00	0.00	0.00	0.50	2.00	0.50
E	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00
F	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00
G	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00
H	3.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00

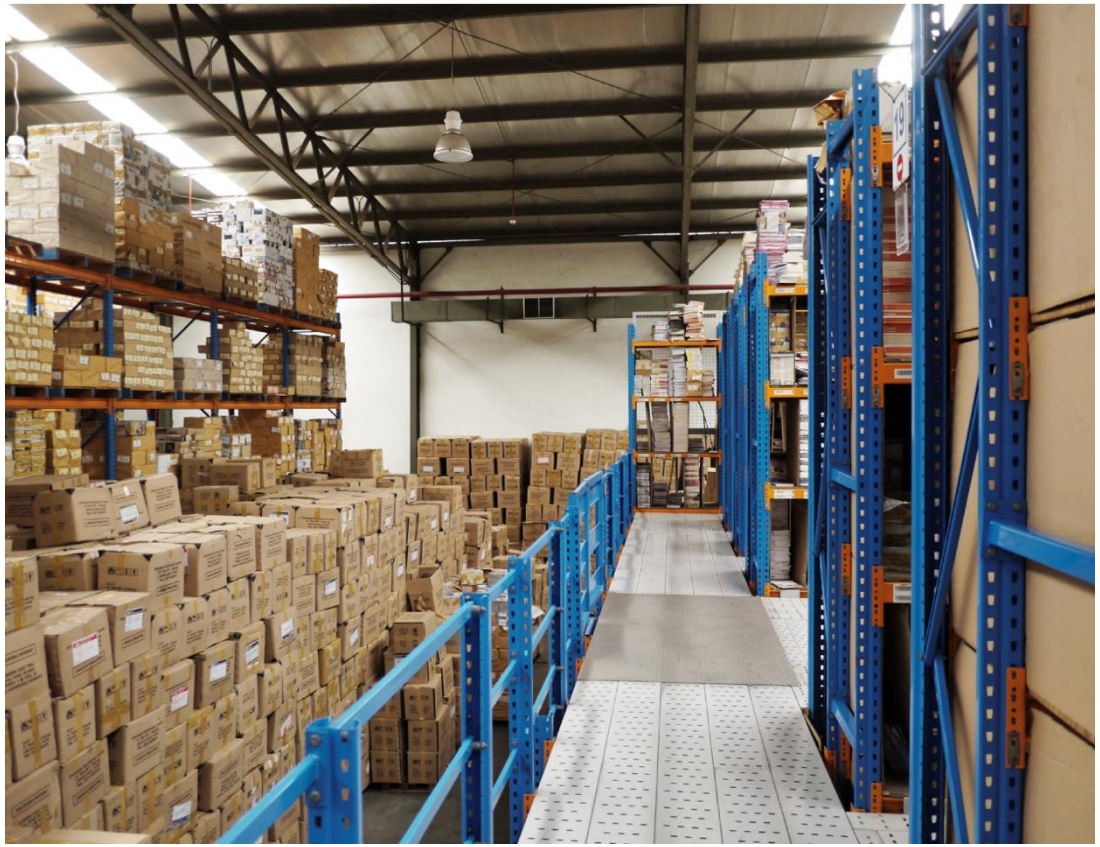


Graphs as matrices

- **Task:** the packages and the connectivity between distributions centres
- **Target:** to predict which warehouses are going to receive lots of packages
- **Graphs as matrices**



	A	B	C	D	E	F	G	H
A	0.00	2.00	1.00	0.50	0.00	0.00	0.00	0.00
B	0.00	0.00	0.00	0.25	3.00	0.00	0.00	0.00
C	0.00	2.00	0.00	4.00	0.00	0.00	0.00	0.00
D	0.00	0.00	0.00	0.00	0.00	0.50	2.00	0.50
E	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00
F	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00
G	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00
H	3.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00



University of Glasgow

Flow analysis: using matrices to model discrete problems

- **Adjacency matrix *A*** with edges being the weights of the **connection** between sites
- **Linear assumption:** the packages arriving at one site is a weighted sum of the packages currently at each of the other sites
- At any point in time, we can write down the proportion of packages at each depot as a vector $\boldsymbol{x}_t \in \mathbb{R}^V$, where *V* is the number of vertices in the graph (number of depots), e.g.

$$\mathbf{x}_t = [0.05 \quad 0.15 \quad 0.30 \quad 0.20 \quad 0.03 \quad 0.12 \quad 0.08 \quad 0.07]$$

  - Vectors, Matrices and linear maps
  - The flow of packages (per day) between depots is a linear map $\mathbb{R}^V \to \mathbb{R}^V$. This is represented by the adjacency matrix $A \in \mathbb{R}^{V \times V}$ (a square matrix).

University of Glasgow

Flow analysis: using matrices to model discrete problems

- This allows us to analyse an apparently *discrete* problem (connectivity of graphs) with tools from *continuous* mathematics (vectors and matrices).
  - e.g. from today to tomorrow

- The advantage of vectorised operations is that they can be accelerated using hardware such as a GPU

- Suppose we start with an *initial distribution* of packages: a vector $\boldsymbol{x_{t=0}}$.

$$\mathbf{x}_{\mathbf{t}=\mathbf{1}} = \mathbf{x}_{\mathbf{t}=\mathbf{0}} A = A^T {\mathbf{x}_{\mathbf{t}=\mathbf{0}}}^T$$

Flow analysis: using matrices to model discrete problems

Given matrix A and vector $\mathbf{x}_t$:

$$\mathbf{x}_t = [0.05 \quad 0.15 \quad 0.30 \quad 0.20 \quad 0.03 \quad 0.12 \quad 0.08 \quad 0.07]$$

and

$$A = \begin{bmatrix} 0.0 & 2.0 & 1.0 & 0.5 & 0.0 & 0.0 & 0.0 & 0.0 \\ 0.0 & 0.0 & 0.0 & 0.25 & 3.0 & 0.0 & 0.0 & 0.0 \\ 0.0 & 2.0 & 0.0 & 4.0 & 0.0 & 0.0 & 0.0 & 0.0 \\ 0.0 & 0.0 & 0.0 & 0.0 & 0.0 & 0.5 & 2.0 & 0.5 \\ 0.0 & 0.0 & 0.0 & 0.0 & 0.0 & 0.0 & 0.0 & 0.0 \\ 0.0 & 0.0 & 0.0 & 0.0 & 0.0 & 0.0 & 0.0 & 0.0 \\ 0.0 & 0.0 & 0.0 & 0.0 & 0.0 & 0.0 & 0.0 & 0.0 \\ 3.0 & 0.0 & 0.0 & 0.0 & 0.0 & 0.0 & 0.0 & 0.0 \end{bmatrix}$$

How many packages will be at each depot tomorrow?

This "**rotates**" the distribution of packages from today to tomorrow.

We'll compute the multiplication:

$$\mathbf{x}_{t+1} = \mathbf{x}_t A$$

To find how many will be in first site $\mathbf{x}_{t+1}(1)$ tomorrow, we can calculate the first element of the resulting vector $\mathbf{x}_{t+1}$:

$$\mathbf{x}_{t+1}(1) = (0.05 \times 0.0) + (0.15 \times 2.0) + (0.30 \times 1.0) + (0.20 \times 0.5) + \ldots = 0.3 + 0.3 + 0.1 = 0.7$$

Other Important questions

There are some harder questions we can ask:

- What about in a week‘s time? What will $x_{t=7}$ be?
- What about in *one hour's* time (i.e. a 24th of a day)? What will be $x_{t=1/24}$be?
- What about at time infinity $x_{t=\infty}$? What is the long term behaviour? Will the system reach a steady state (an **equilibrium**)? Or will it oscillate forever?
- What about if we wanted to go backwards in time? If we know $x_{t=0}$, can we predict yesterday $x_{t=-1}$?

We will solve these problems today, using some new operations that we can do with *certain kinds* of matrices:

University of Glasgow

Using matrices to model discrete problems

- packages moving between depots (how many packages at each depot at an instant in time)
- users moving between web pages (how many users on each webpage)
- cancer cells moving between tumour sites (how many cancer cells at each tumour site)
- trade between states (how many items in each state)
- shoppers walking between retailers (how many shoppers in each shop)
- traffic across a load-balancing network (how many cars at each junction)
- NPCs (non-player characters) moving between towns in a game (how many NPCs in each town)
- fluid moving between regions (how much fluid in each tank)
- blood flowing between organs (how much blood in each organ)
- beliefs moving among hypotheses (how much we believe in each hypothesis)

University of Glasgow

New matrix operations

- Matrices can be **exponentiated**: $\boldsymbol{C}=\boldsymbol{A}^n$; this "repeats" the effect of matrix (e.g. $\boldsymbol{C}=\boldsymbol{AAAA}$)
- Matrices can be **inverted**: $\boldsymbol{C}=\boldsymbol{A}^{-1}$; this undoes the effect of a matrix
- We can find **eigenvalues**: $\boldsymbol{Ax}=\boldsymbol{\lambda x}$; this identifies specific vectors x that are only scaled by a factor λ (not rotated) when transformed by matrix A.
- Matrices can be **factorised**: $\boldsymbol{A}=\boldsymbol{U\Sigma V}^T$; any matrix can expressed as the product of three other matrices with special forms.
- We can measure some properties of $\boldsymbol{A}$ numerically, including the **determinant**, **trace** and **condition number**.

University of Glasgow

Matrix powers (exponentiation)

- We can now define $A^2 = AA, A^3 = AAA, A^4 = AAAA$, etc

- These are the **powers** of a matrix, and are only defined for square matrices
  - Otherwise, we'd change the dimensions after the first step and be unable to reapply the same matrix

- **Package distribution**: What about in a week‘s time? What will $x_{t=7}$ be?
  - We can simply apply the matrix seven times; raising it to the power of 7.

Matrix powers (exponentiation)

- numpy.linalg.matrix_power(a, n)

```
>>> from numpy.linalg import matrix_power
>>> i = np.array([[0, 1], [-1, 0]]) # matrix equiv. of
>>> matrix_power(i, 3) # should = -i
array([[ 0, -1],
       [ 1,  0]])
>>> matrix_power(i, 0)
array([[1, 0],
       [0, 1]])
>>> matrix_power(i, -3) # should = 1/(-i) = i, but w/ f
array([[ 0.,  1.],
       [-1.,  0.]])
```

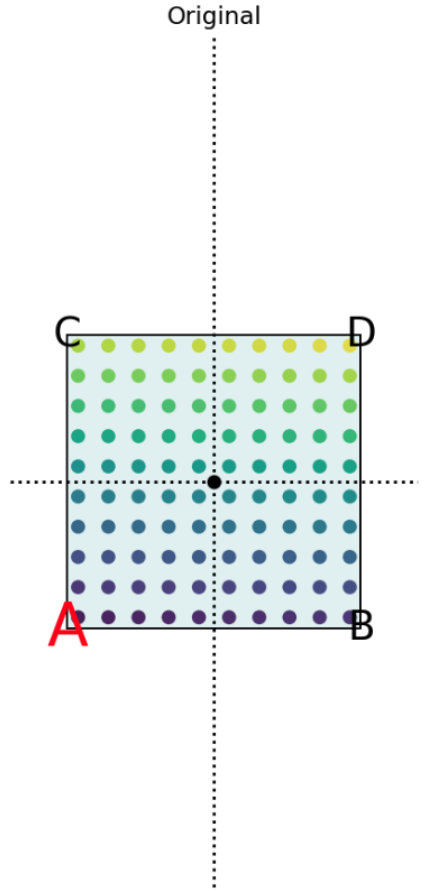
- Applying a matrix multiple times

```
# Define a function that raises a matrix
def powm(A, n):
    B = np.eye(A.shape[0]) # start with
    for i in range(n):
        B = A @ B  # @ performs matrix m
    return B
```

```
# Demonstrate raising a matrix A to powe
A = np.array([[1.5, 0.0], [-1.1, 0.1]])
print_matrix("A", A)

for i in range(5):
    print_matrix("A^%d" % i, powm(A, i))
```

```
A
 [[ 1.5  0. ]
 [-1.1  0.1]]
A^0
 [[1. 0.]
 [0. 1.]]
A^1
 [[ 1.5  0. ]
 [-1.1  0.1]]
A^2
 [[ 2.25  0.  ]
 [-1.76  0.01]]
A^3
 [[ 3.38  0.  ]
 [-2.65  0.  ]]
A^4
 [[ 5.06  0.  ]
 [-3.98  0.  ]]
```

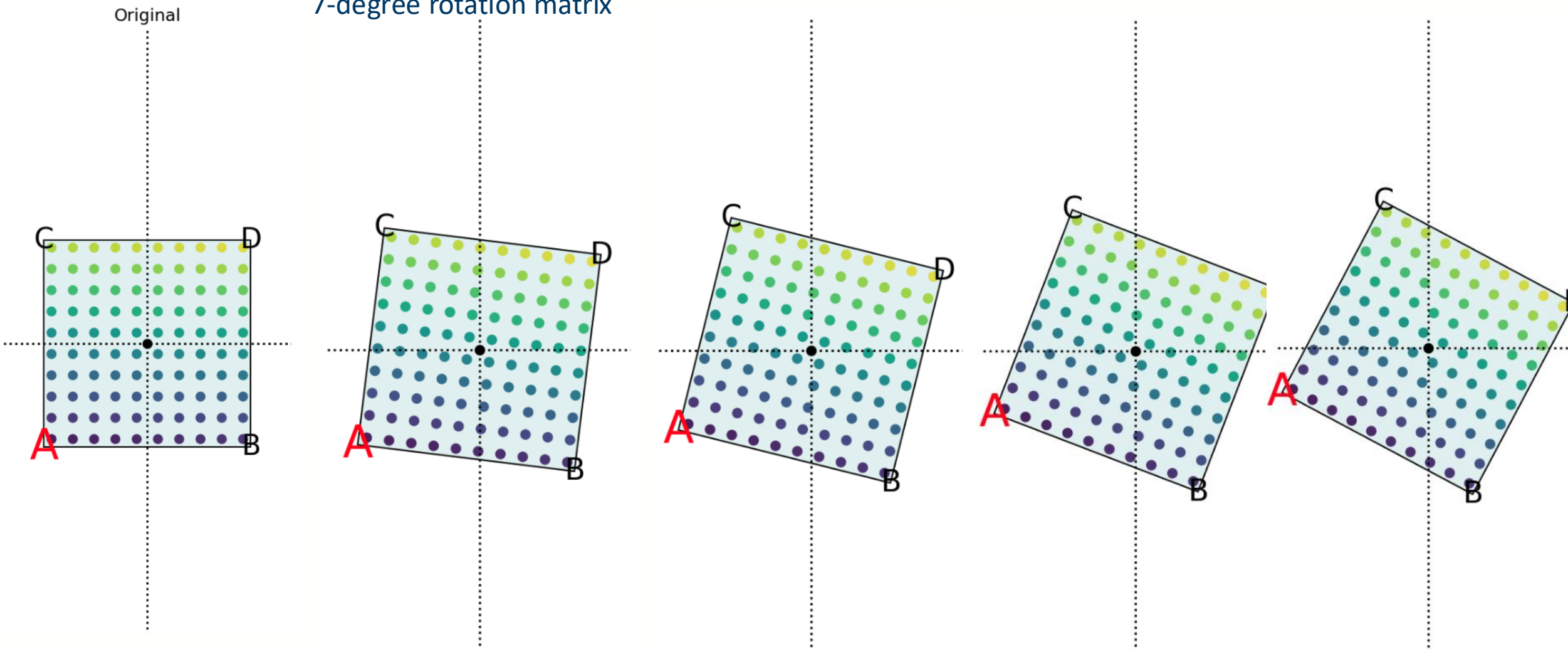


Matrix powers

- Original

A^1 A^2 A^3 A^4

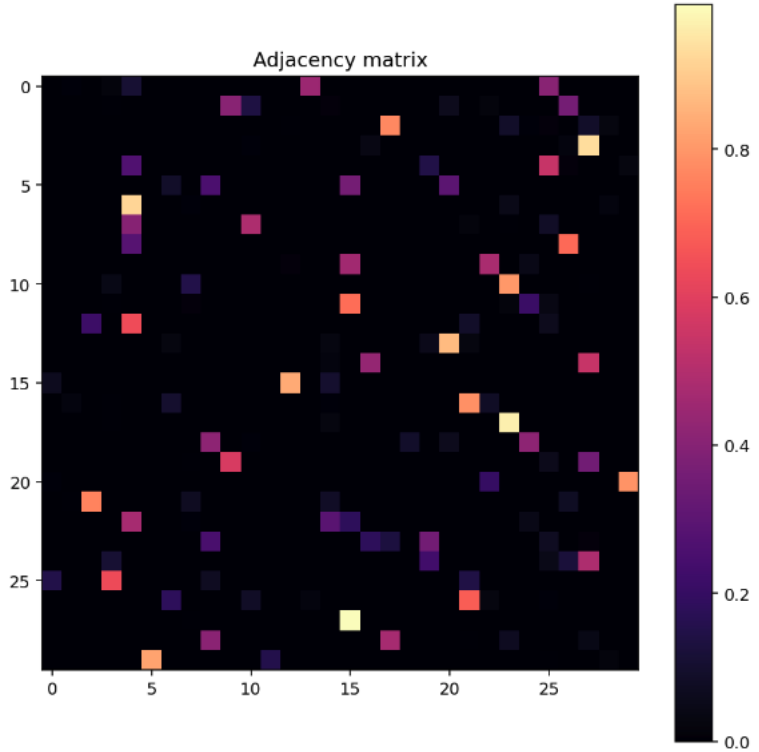
7-degree rotation matrix



Stable point: all roads lead to Rome

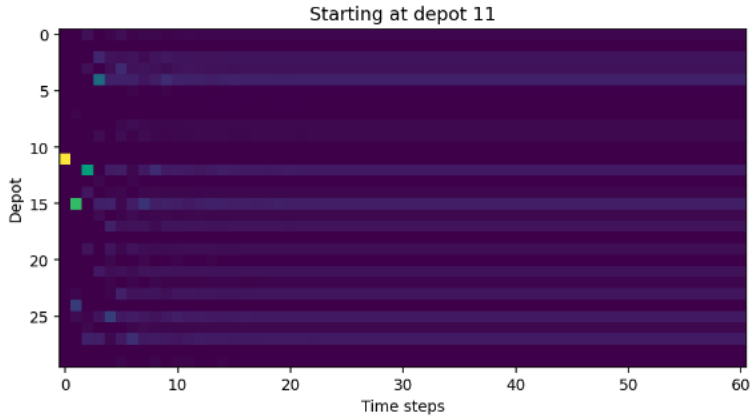
- A simple experiment: Imagine we have a conserving adjacency matrix modelling the flow of packages around a network of depots.

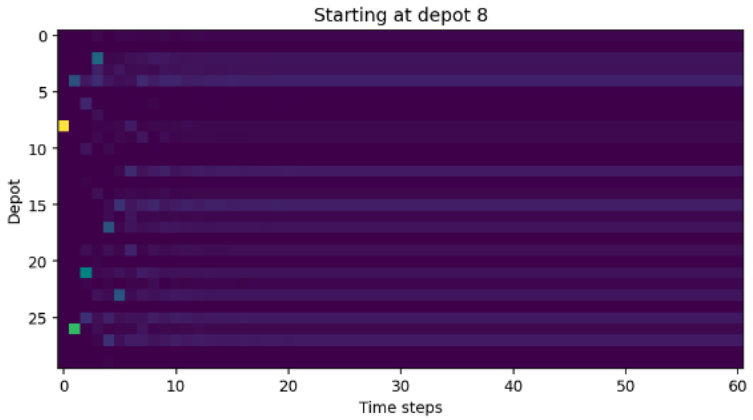
- If we start with different package distributions and then let the "natural" flow begin, what will happen?

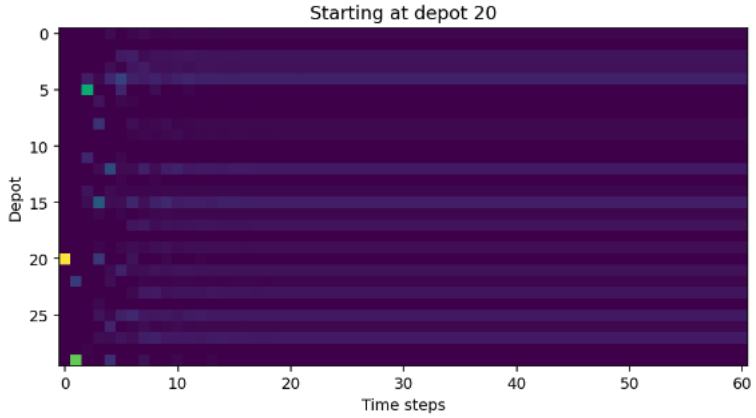


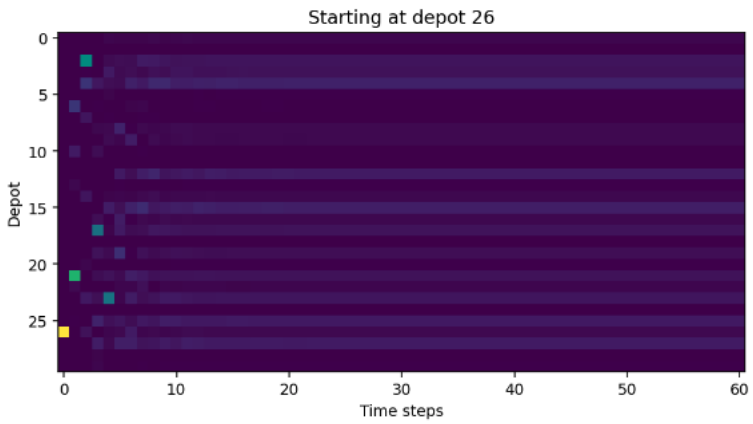
Stable point: all roads lead to Rome

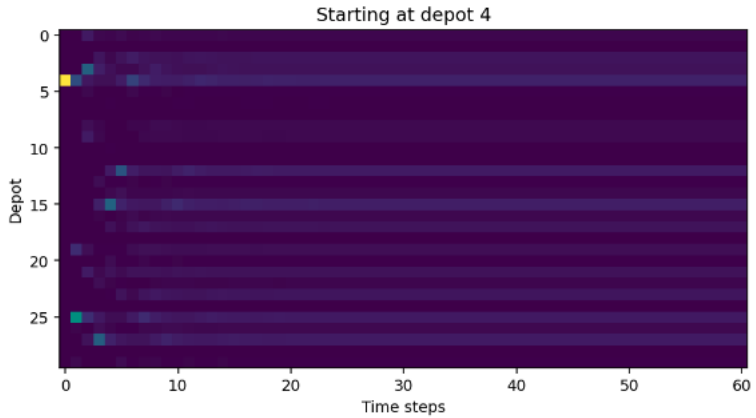
- Run the experiment a few times with 5 different starting states (11, 8, 20, 26, 4)











Surprised...?

University of Glasgow

Stable point: all roads lead to Rome

- No matter what the initial package distribution is, after several days the distribution will settle down to a **steady state** (or stable point).

- This vector is one of the *eigenvectors* of the adjacency matrix.

- Applying the matrix again does not change the distribution of the packages, i.e. $\mathbf{x}_{t=60}\, \boldsymbol{A} = \mathbf{x}_{t=59}\boldsymbol{A}$

University of Glasgow

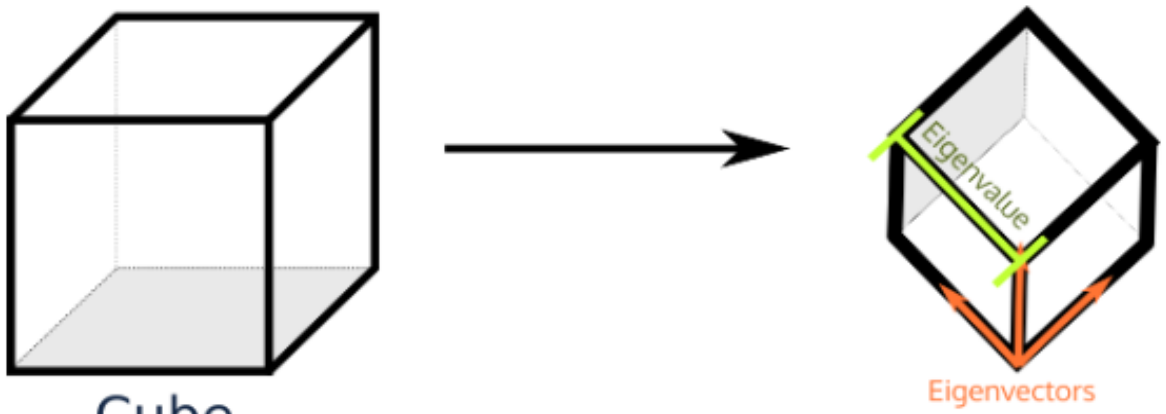
Eigenvalues and eigenvectors

- A matrix represents a special kind of function: a **linear transform**; an operation that performs **rotation** and **scaling** on vectors.

- However, there are certain vectors which don't get rotated when multiplied by the matrix.
  - They only get scaled (stretched or compressed).
  - These vectors are called **eigenvectors**, and they can be thought of as the "fundamental" or "characteristic" vectors of the matrix.
  - The scaling factors that the matrix applies to its eigenvectors are called **eigenvalues**.

Eigenvalues and eigenvectors

- We can visualise the effect of a matrix transformation by imagining a parallelepiped (whose edges are vectors) being rotated, stretched and compressed.
- If the edges of the parallelelpiped are the **eigenvectors** of the matrix, the parallelepiped will **only** be stretched or compressed, **not** rotated.
- If the eigenvalue is greater than 1, the eigenvector is stretched. If it's between 0 and 1, it's shrunk.
- If the edges of this parallelepiped have unit length, then after the transformation their lengths will be equal to the eigenvalues.



University of Glasgow

How to find the leading eigenvector: the power iteration method

- "All roads lead to Rome", illustrates how to find one of the eigenvectors of a matrix.
  - We simply apply the matrix repeatedly to a random initial vector and wait until the result converges to a steady state.

$$\begin{aligned}\mathbf{x_n} &= AAAA\ldots AA\mathbf{x_0} \\ &= A^n\mathbf{x_0}\end{aligned}$$

  - this **will generally either explode in value or collapse to zero**. However, we can fix the problem by **normalising** the resulting vector after each application of the matrix:

$$\mathbf{x_n} = \frac{A\mathbf{x_{n-1}}}{\|A\mathbf{x_{n-1}}\|_\infty}$$

  - This process is called **power iteration**.

University of Glasgow

How to find the leading eigenvector: the power iteration method

- Regardless of which vector x_0 we start with (the vector above is chosen randomly), the power iteration method always approaches a fixed vector (though possibly with sign flips).
  - This is true for almost every square matrix.
  - The vector that results from power iteration is known as the **leading eigenvector**.
- It satisfies the definition of an eigenvector because the matrix ***A*** performs only scaling on this vector (no rotation).
  - The scaling effect is eliminated by the normalisation step in the power iteration, but any other effects pass through
  - We can write the scaling effect of the ***A*** on an eigenvector ***x*** as follows:

$$\boldsymbol{Ax=\lambda x}$$

where $\boldsymbol{\lambda}$ is the eigenvalue.

Computing eigenvectors and eigenvalues with Numpy

- The power iteration method enables us to calculate the leading eigenvector and eigenvalue, but if we want to know **all** the (linearlly independent) eigenvectors and eigenvalues of a matrix, we can use ***np.linalg.eig***:

- For very large matrices, if you just want to compute the leading eigenvector, power iteration is much faster than using ***np.linalg.eig.***

```
# Compute eigenvalues and eigenvectors with Numpy
print("A=\n%s\n" % A)
evals, evecs = np.linalg.eig(A)  # evecs is a matrix whose
#columns are the unit eigenvectors
print("Eigenvalues:\n", evals)
print()
print("Unit Eigenvectors (columns):\n", evecs)
```

```
A=
[[-1.2011307   1.68673513  2.5605476 ]
 [ 1.68673513  1.70214986  1.81661604]
 [ 2.5605476   1.81661604  0.09588057]]

Eigenvalues:
 [ 4.37541523 -3.20636073 -0.57215476]

Unit Eigenvectors (columns):
 [[-0.46584035 -0.80396258  0.36964433]
 [-0.67928143  0.05720192 -0.73164518]
 [-0.56707099  0.59192238  0.57276382]]
```

Eigenvectors and eigenvalues

- Consider a vector function ***f(x)***. There may exist vectors such that ***f(x)=λx***. The function maps these vectors to scaled versions of themselves. No rotation or skewing is applied, just pure scaling.
- Any square matrix ***A*** represents a function ***f(x)*** and may have vectors like this, such that

$$\boldsymbol{A x_i = \lambda_i x_i}$$

- Each vector $\boldsymbol{x_i}$ satisfying this equation is known as an **eigenvector** and each corresponding factor $\boldsymbol{\lambda_i}$ is known as an **eigenvalue**.
  - For any matrix, the **eigenvalues** are uniquely determined, but the eigenvectors are not.
  - The eigenvectors are **orthogonal**, i.e. the dot product of any pair of eigenvectors is zero.
  - They define a new **coordinate system** (or basis) in which the transformation *A* becomes diagonal — meaning it **only stretches or compresses** along those axes.

University of Glasgow

Compute Eigenvectors and Eigenvalues Analytically

1. Start with the definition: $\mathbf{Av} = \lambda\mathbf{v}$
2. Rearrange it: $\mathbf{Av} - \lambda\mathbf{v} = \mathbf{0} \rightarrow \mathbf{Av} - \lambda\mathbf{Iv} = 0 \rightarrow (\mathbf{A} - \lambda\mathbf{I})\mathbf{v} = \mathbf{0}$
   - Since **v** must be non-zero, it follows that $\mathbf{A} - \lambda\mathbf{I}$ must be *singular*
   - Therefore that $\det(\mathbf{A} - \lambda\mathbf{I}) = \mathbf{0}$
   - The roots of this equation will give us the eigenvalues $\lambda_1, \lambda_2, \ldots$
   - Substitute each calculated eigenvalue back into the system and solve by row reduction (or otherwise): $(\mathbf{A} - \lambda_i\mathbf{I})\mathbf{v_i} = \mathbf{0}$
     - This system will have infinitely many solutions – it's singular after all
3. The solution will form an eigenplane (or eigenline) any non-zero vector on this is an eigenvector.
   - For simplicity, we often normalise the eigenvectors (→ unit length)

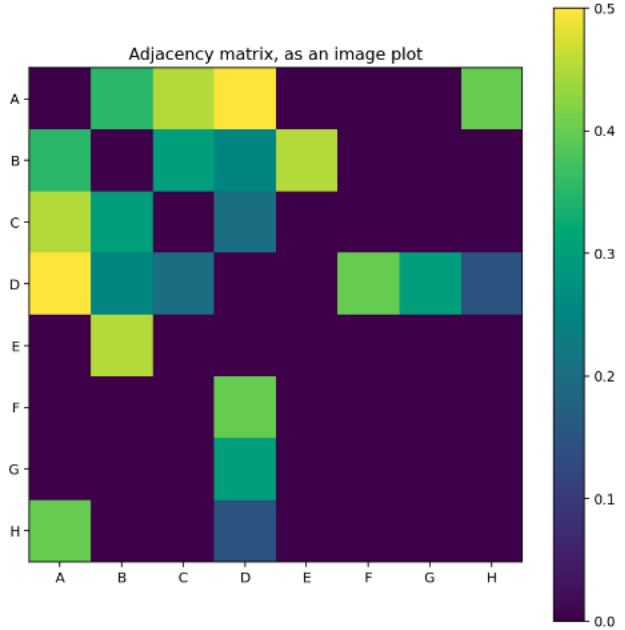
Eigenproblems

- **Eigenproblems** are problems that can be tackled using eigenvalues and eigenvectors.

- E.g. Revisiting the package distribution problem
  - What about at time infinity $x_{t=\infty}$? What is the long-term behaviour? Will the system reach a steady state (an **equilibrium**)? Or will it oscillate forever?

Revisiting the package distribution problem

- We will work with an undirected (symmetric) graph to ensure that the eigenvalues are all real (why?)
  - Making the adjacency matrix symmetric also means we don't have to worry about transposing it before using ***np.linalg.eig***



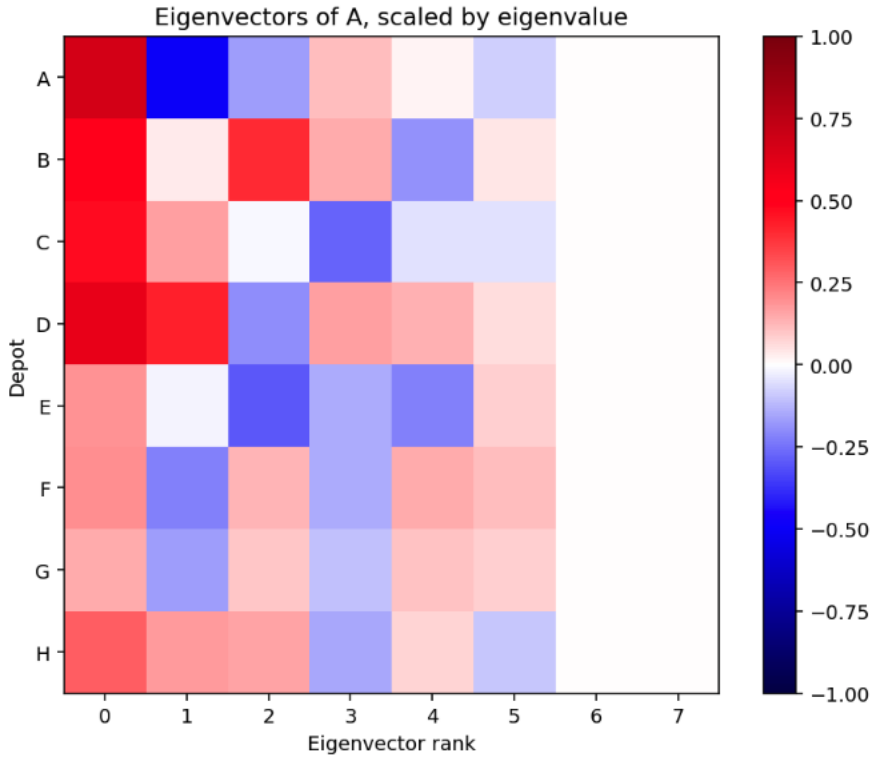
Revisiting the package distribution problem

- compute eigendecomposition of A

```
# compute eigendecomposition of A
evals, evecs = np.linalg.eig(A)
print(evals)

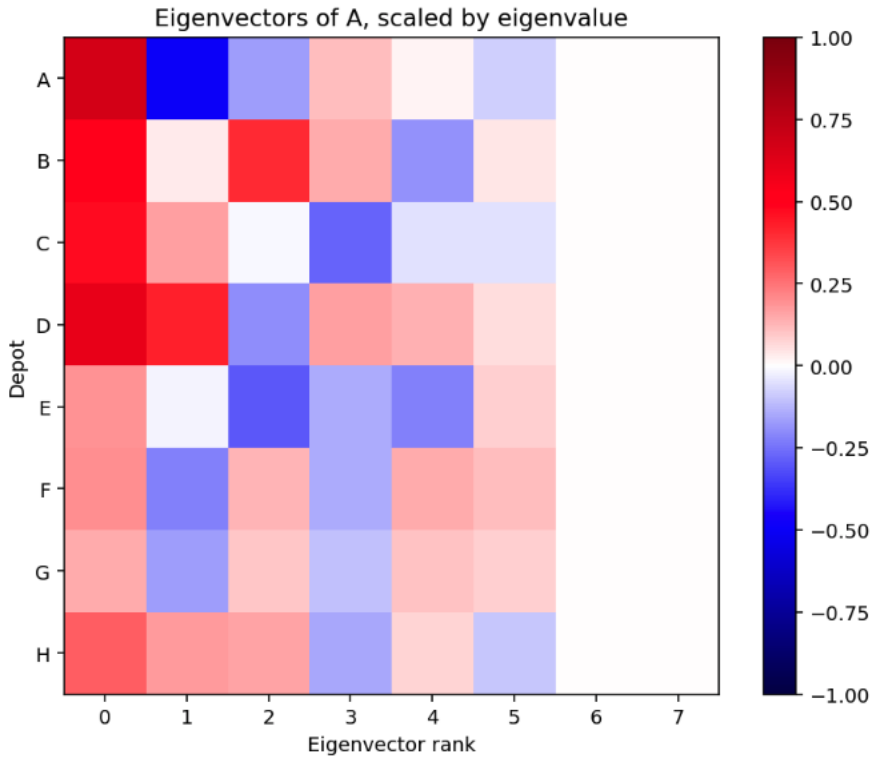
# plot eigenvectors as an image, in order of decreasing eigenvalue
fig = plt.figure(figsize=(8, 6))
ax = fig.add_subplot(1,1,1)
order = np.argsort(-np.abs(evals))
img = ax.imshow(((evecs[:, order] * evals[order])), cmap='seismic', vmin=-1, vmax=1)
ax.set_yticklabels("ABCDEFGH")
ax.set_yticks(np.arange(8))
ax.set_xlabel("Eigenvector rank")
ax.set_ylabel("Depot")
fig.colorbar(img)#, orientation='horizontal')
ax.set_title("Eigenvectors of A, scaled by eigenvalue");
```

```
[ 1.20818368e+00 -7.51883610e-01 -6.07983833e-01 -4.59054268e-01
  2.24475250e-01  3.86262784e-01 -1.35946555e-17  1.83665339e-17]
```



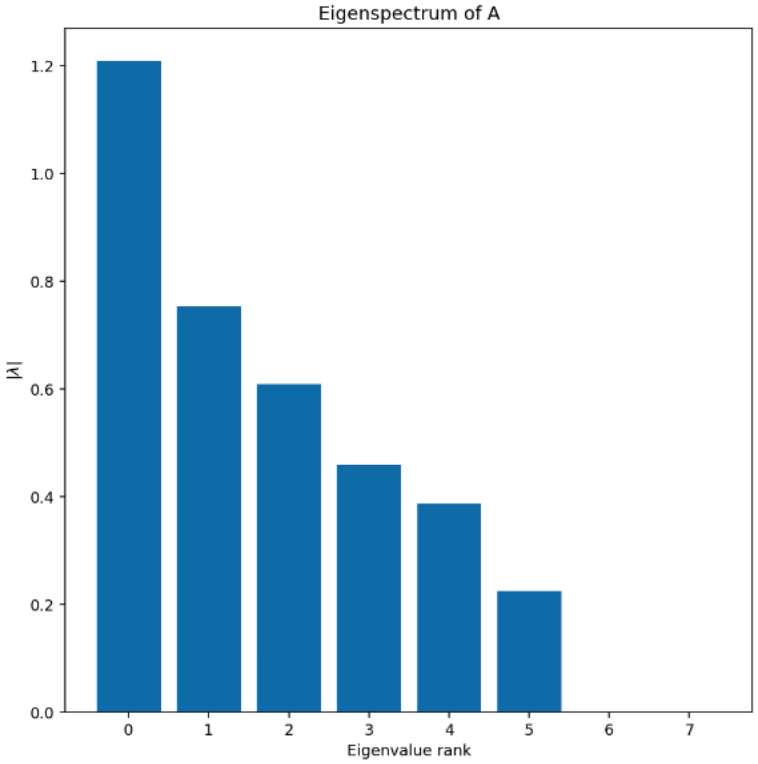
Revisiting the package distribution problem

- The leading eigenvector tells us what the steady state of the package distribution will be if we keep the system running for long enough.
- Small eigenvalues correspond to route patterns that are rarely used, and large eigenvalues to the dominant path of packages moving through the system.
- For this distribution network, most of the packages will pile up at A, B, C, D and H.



The eigenspectrum

- The eigenspectrum is just the sequence of absolute eigenvalues, ordered by magnitude $|\lambda_1|>|\lambda_2|>\cdots>|\lambda_n|$.
- This *ranks* the eigenvectors in order of "importance".
  - This can be useful in finding "simplified" versions of linear transforms.



University of Glasgow

Numerical instability of eigendecomposition algorithms

- *np.linalg.eig* can suffer from numerical instabilities due to rounding errors resulting from limitations on floating point precision.
- This means that sometimes the smallest eigenvectors are not completely orthogonal.

- If your matrix satisfies certain special conditions, you might be able to use a more stable algorithm. For example, if it is real and symmetric (or Hermitian, in the case of a complex matrix), you can use *np.linalg.eigh*.
  - a **Hermitian matrix** is a complex square matrix that is equal to its own conjugate transpose

np.linalg.eig **vs** np.linalg.eigh

- np.linalg.eig
- np.linalg.eigh

numpy.linalg.eig

`linalg.eig(a)` [source]

Compute the eigenvalues and right eigenvectors of a square array.

Parameters: **a** : *(..., M, M) array*

Matrices for which the eigenvalues and right eigenvectors will be computed

Returns: **A namedtuple with the following attributes:**

eigenvalues : *(..., M) array*

The eigenvalues, each repeated according to its multiplicity. The eigenvalues are not necessarily ordered. The resulting array will be of complex type, unless the imaginary part is zero in which case it will be cast to a real type. When *a* is real the resulting eigenvalues will be real (0 imaginary part) or occur in conjugate pairs

eigenvectors : *(..., M, M) array*

The normalized (unit "length") eigenvectors, such that the column `eigenvectors[:,i]` is the eigenvector corresponding to the eigenvalue `eigenvalues[i]`.

Raises: **LinAlgError**

If the eigenvalue computation does not converge.

numpy.linalg.eigh

`linalg.eigh(a, UPLO='L')` # [source]

Return the eigenvalues and eigenvectors of a complex Hermitian (conjugate symmetric) or a real symmetric matrix.

Returns two objects, a 1-D array containing the eigenvalues of *a*, and a 2-D square array or matrix (depending on the input type) of the corresponding eigenvectors (in columns).

Parameters: **a** : *(..., M, M) array*

Hermitian or real symmetric matrices whose eigenvalues and eigenvectors are to be computed.

UPLO : *{'L', 'U'}, optional*

Specifies whether the calculation is done with the lower triangular part of *a* ('L', default) or the upper triangular part ('U'). Irrespective of this value only the real parts of the diagonal will be considered in the computation to preserve the notion of a Hermitian matrix. It therefore follows that the imaginary part of the diagonal will always be treated as zero.

Returns: **A namedtuple with the following attributes:**

eigenvalues : *(..., M) ndarray*

The eigenvalues in ascending order, each repeated according to its multiplicity.

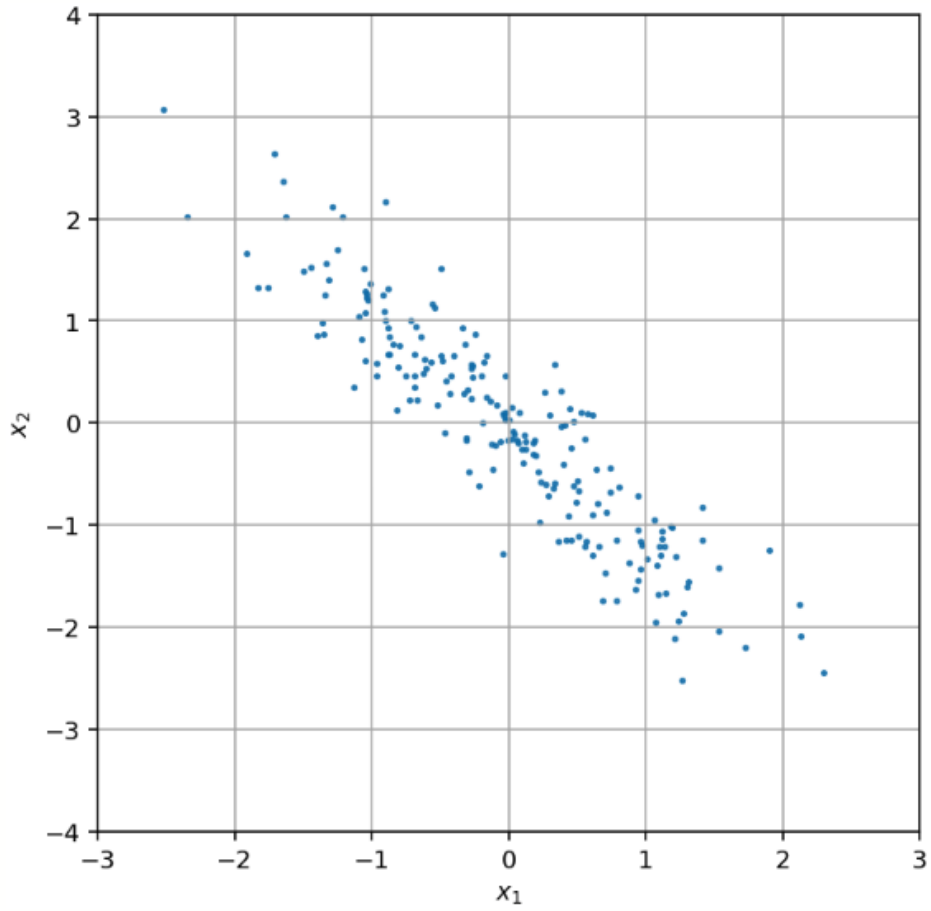
eigenvectors : *{(..., M, M) ndarray, (..., M, M) matrix}*

The column `eigenvectors[:, i]` is the normalized eigenvector corresponding to the eigenvalue `eigenvalues[i]`. Will return a matrix object if *a* is a matrix object.

Principal Component Analysis (PCA) - analysing data with linear matrices

- Let's generate a dataset in $\mathbb{R}^2$:
  - 200 examples
  - First 10 observations

```
[[-1.36401964  0.97382507]
 [-0.67846501  0.94091341]
 [-0.26893673  0.56671007]
 [ 0.32968114 -0.63947773]
 [ 0.63723353 -0.45517774]
 [-0.26585839  0.44739262]
 [ 0.33394489 -0.59711092]
 [ 0.9658487  -1.16179343]
 [ 0.9424511  -0.71066843]
 [ 0.7834083  -1.74599991]]
```



University of Glasgow

Principal Component Analysis (PCA) - analysing data with linear matrices

- **Mean vector:** The mean vector is the **geometric centroid** of a set of vectors and can be thought of as capturing "centre of mass" of those vectors.

$$\mathbf{mean}(\mathbf{x_1}, \mathbf{x_2}, \ldots, \mathbf{x_n}) = \frac{1}{N} \sum_i \mathbf{x_i}$$

- **Covariance matrix:** measures the spread of a dataset.

$$\Sigma_{ij} = \frac{1}{N-1} \sum_{k=1}^{N} (X_{ki} - \mu_i)(X_{kj} - \mu_j)$$

Covariance Matrix Example

Dataset *T*:

Height	Weight	Mark
165.2000	58.1000	75.0000
178.5000	72.3000	88.0000
155.0000	51.5000	62.0000
170.8000	65.0000	91.0000
183.1000	80.5000	78.0000
160.5000	55.2000	68.0000
175.0000	69.8000	85.0000
158.3000	53.0000	95.0000
168.9000	62.1000	70.0000
180.2000	75.6000	82.0000
152.4000	48.9000	73.0000
173.5000	67.5000	65.0000
162.8000	56.7000	90.0000
177.0000	71.1000	77.0000
166.5000	60.0000	84.0000



Covariance matrix Σ:

	Height	Weight	Mark
Height	89.4198	88.6014	19.8519
Weight	88.6014	89.4755	17.0862
Mark	19.8519	17.0862	101.1238

- **Diagonal entries**: variances within the features
- **Other entries**: co-variances between the feature pairs
- **Notice that**:
  - Height-Weight pairs have a high covariance
  - Height-Mark and Weight-Mark have low covariance
- Also, **Σ is symmetric** (by definition)

University of Glasgow

Principal Component Analysis (PCA)

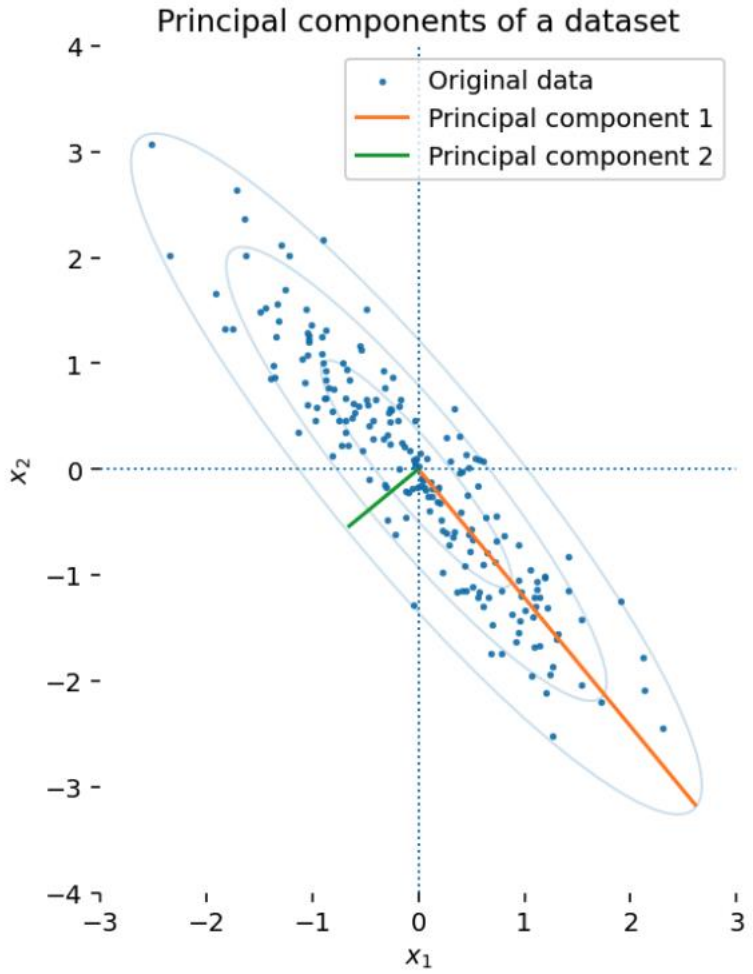
Process:

1. Standardise the data (mean = 0, standard deviation = 1). (Optional, but recommended)
2. Compute the covariance matrix.
3. Perform eigendecomposition on the covariance matrix.
4. Sort eigenvalues in descending order and choose the top k eigenvectors, where k is the number of dimensions you want to keep.
5. Transform the original data using these k eigenvectors to get the principal components.

See lecture notes for an extensive example.

Principal Component Analysis (PCA)

- The eigenvectors of the covariance matrix are called the **principal components**
  - They tell us the directions in which the data varies most
- This is **an incredibly useful thing to be able to do**, particularly with high-dimensional data sets where the variables may be correlated in complicated ways.



University of Glasgow

Eigendecomposition of the covariance matrix into its eigenvectors and eigenvalues

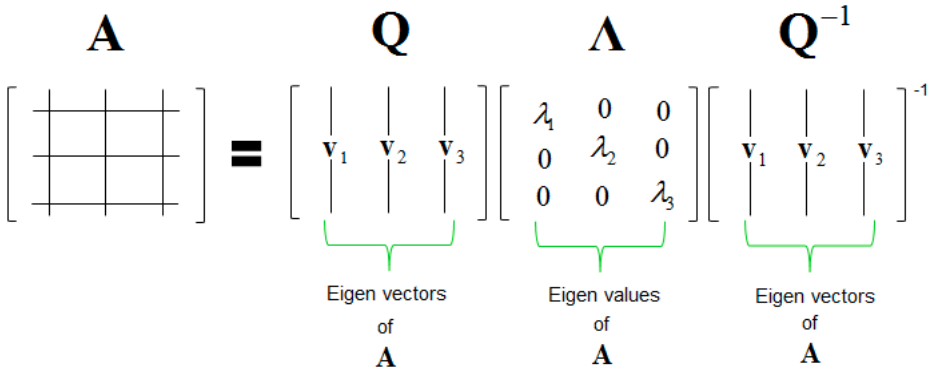
- We are now interested in reconstruct the covariance matrix using its eigenvectors and eigenvalues.

$$\Sigma = Q\Lambda Q^T$$

where Q is a matrix of unit eigenvectors $\mathbf{x_i}$ (same as the output np.linalg.eig) and Λ is a diagonal matrix of eigenvalues (λ_i on the diagonal, zero elsewhere).

Eigendecomposition

- **Eigendecomposition** is the factorisation of a matrix into a canonical form, whereby the matrix is represented in terms of its eigenvalues and eigenvectors.



- Decomposition does **NOT** result in a *compression* of the matrix; **instead**, it breaks it down into constituent parts to make certain operations on the matrix easier to perform.

University of Glasgow

Approximating a matrix

- Imagine we started with a very high dimensional data set, so $\boldsymbol{A}$ is a very large matrix.
  - It's so large, we don't want to store it in memory.

- Instead, we just want to store the first few principal components $\boldsymbol{Q}$ and use these to reconstruct an *approximation* to $\boldsymbol{A}$. Providing we keep the largest principal components, we will probably retain most of the information.

- Matrix approximation can be used to simplify transformations or to compress matrices for data transmission.

Eigendecomposition and Reduction Toy Example

1

Covariance matrix Σ:

	Height	Weight	Mark
Height	89.4198	88.6014	19.8519
Weight	88.6014	89.4755	17.0862
Mark	19.8519	17.0862	101.1238

2

- Eigenvectors and eigenvalues of Σ:

Eigenvectors V / PCs =

0.6780	0.1935	0.7092	← *Height*
0.6738	0.2222	-0.7047	← *Weight*
0.2939	-0.9556	-0.0203	← *Mark*

Eigenvalues E =

186.0809 → 1st PC: 66.45% of total variance

93.1318 → 2nd PC: 33.26% of total variance

0.8065. → 3rd PC: 0.29% of total variance

3

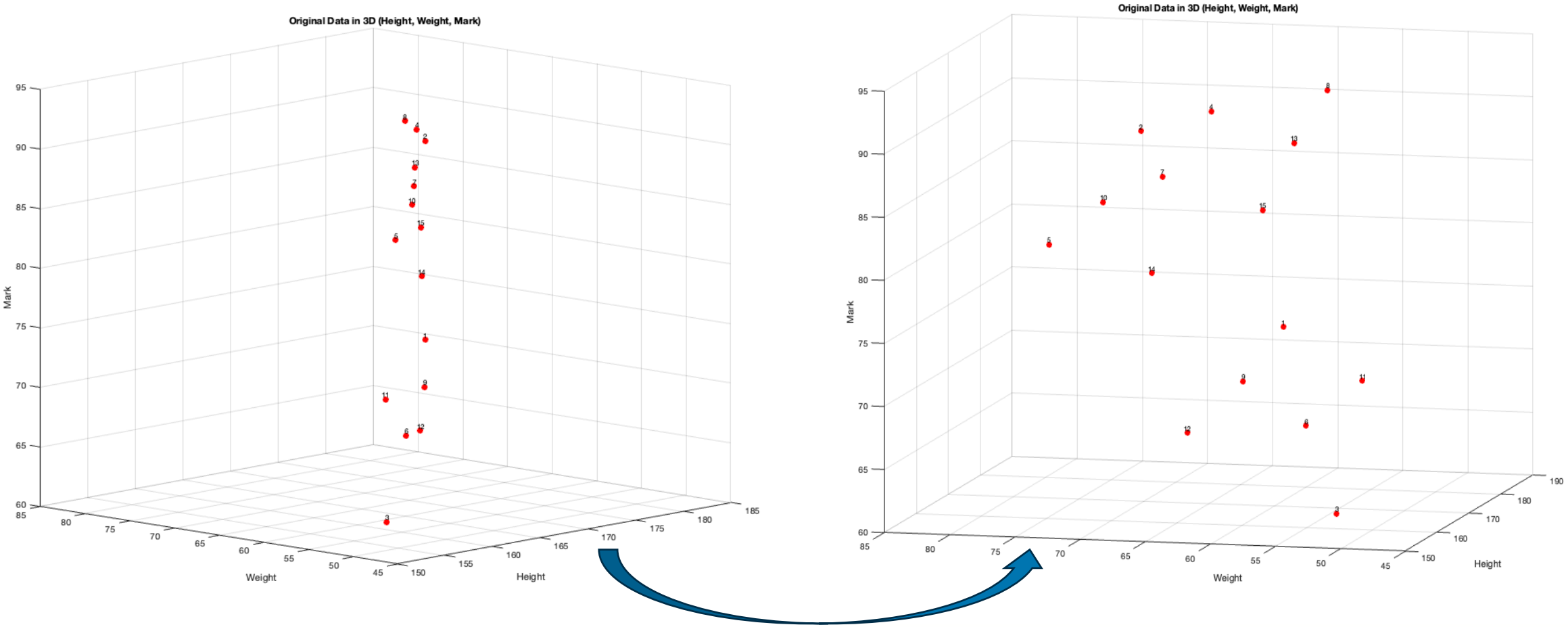
- $\rightarrow k = 2$ PCs are enough
- Reduced dataset R:
  - $R = T_{\text{centred}} \times V_{k=2}$
  - Sanity check:
    - $T \in \mathbb{R}^{15\times3}$, $V_{k=2} \in \mathbb{R}^{3\times2}$
    - $\rightarrow R \in \mathbb{R}^{15\times2}$ → Reduced!

- Total variance: $\sum \lambda_i = 280.0191$
- Each λ_i indicates the proportion of the corresponding vector to the total variance
  - i.e., the relative importance of the vector in describing the data

Eigendecomposition and Reduction Toy Example

Original data in 3D

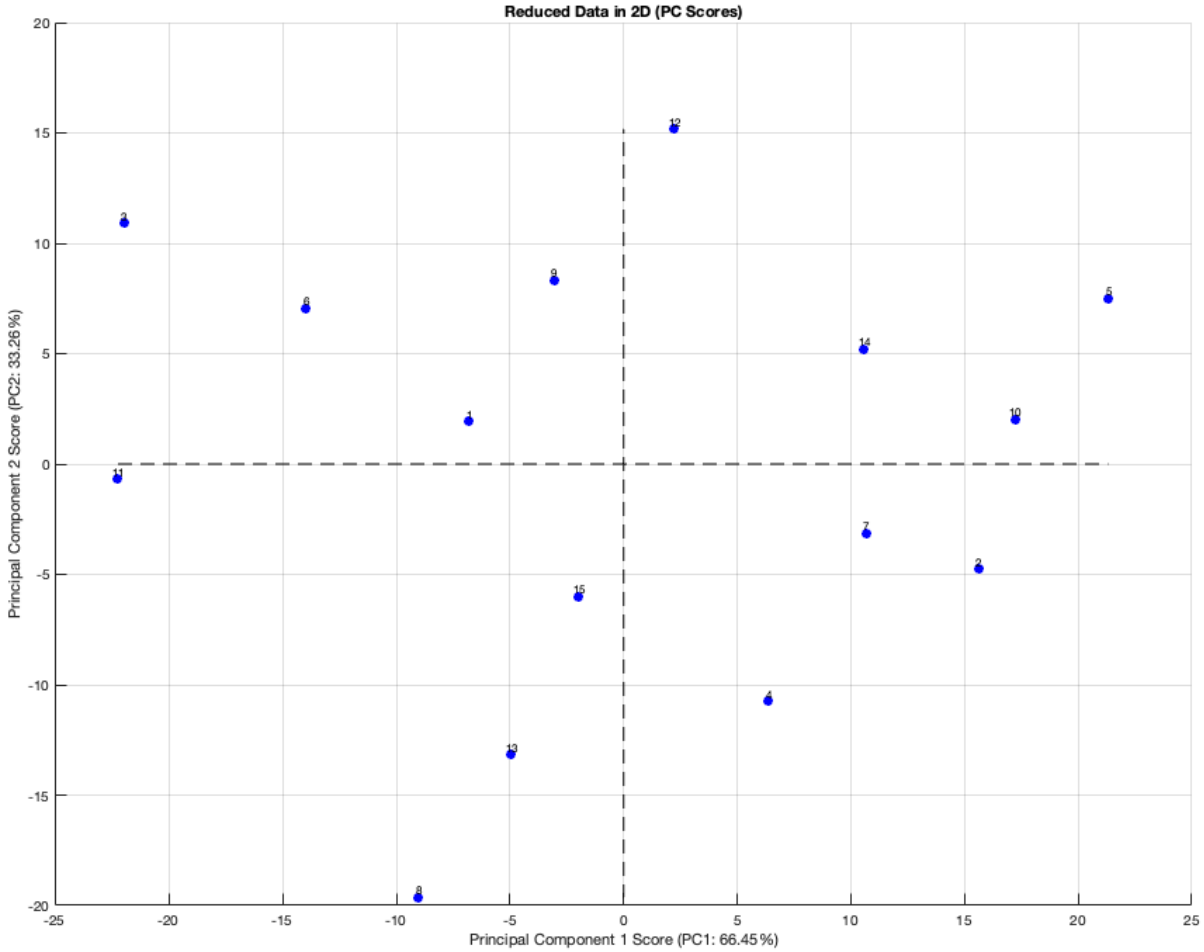
It’s visible that score does not correlate with height or weight.



Eigendecomposition and Reduction Toy Example

Reduced data in 2D

Variance highlighted by transforming to the new PC axis



Eigendecomposition and the Eigenspectrum

- **Eigendecomposition**: factorisation of a matrix into its constituent eigenvalues and eigenvectors
- **Eigenspectrum**: the sequence of absolute eigenvalues, *ranks* the eigenvectors in order of "importance”
  - One large eigenvalue and many small ones - just one vector might approximate this matrix.
  - All eigenvalues similar magnitude? We will not be able to approximate this transform easily.

University of Glasgow

Eigendecomposition VS PCA

- **Eigendecomposition** is the factorisation of a matrix into its eigenvalues and eigenvectors. Specifically, for a square matrix.
  - decompose a matrix into its constituent eigenvalues and eigenvectors.
  - *a general matrix decomposition method*

- **PCA** is a statistical procedure that uses eigendecomposition (or singular value decomposition) to convert correlated features (**covariance matrix**) into a set of linearly uncorrelated features called principal components.
  - *a data analysis technique*
  - dimensionality reduction
  - noise reduction and data visualisation.
  - discover which dimensions (features) capture the most variance in the data.

University of Glasgow

Uses of eigendecomposition

- **Matrix decomposition** (e.g. PCA) is an *essential* tool in data analysis.
  - Recommenders, image compression, visualisation etc.
- **Spectral Clustering:** Used for graph-based clustering.
  - The graph Laplacian matrix's eigenvectors are used to cluster data points
- **PageRank Algorithm:** Used by Google's search engine to rank web pages.
  - Relies on finding the eigenvector corresponding to the largest eigenvalue of the modified adjacency matrix of the web graph.
- **Latent Semantic Analysis (LSA):** A text mining technique for topic modeling and dimensionality reduction.
  - Uses singular value decomposition (a generalization of eigendecomposition) of the term-document matrix to capture latent topics.

Matrix Properties related to Eigendecomposition

- **Trace:** The trace of a square matrix can be computed from the sum of its diagonal values:

$$\mathrm{Tr}(A) = a_{1,1} + a_{2,2} + \cdots + a_{n,n}$$

- The sum of the variances (for a covariance matrix) in the original basis → *total variance*
- It is also equal to the sum of the eigenvalues of ***A***

$$\mathrm{Tr}(A) = \sum_{i=1}^{n} \lambda_i$$

- **Determinant:** is equal to the product of the eigenvalues of the matrix.

$$\det(A) = \prod_{i=1}^{n} \lambda_i$$

- It measures how much the space expands or contracts after the linear transform. (the **volume** of the parallelepiped formed by the transformed basis vectors)
- If any eigenvalue is 0, det(A)=0.
  - the transformation **is non-invertible / singular**
  - Dimension has collapsed → information has been lost.

Definite and semi-definite matrices

A matrix is called

- **positive definite** if all of its eigenvalues are greater than zero: $\lambda_i > 0$.
- **positive semi-definite** if all of its eigenvalues are greater than or equal to zero: $\lambda_i \geq 0$.
- **negative definite** if all of the eigenvalues are less than zero: $\lambda_i < 0$,
- **negative semi-definite** if all the eigenvalues are less than or equal to zero: $\lambda_i \leq 0$.

- A covariance matrix is **symmetric**, and its entries are based on the variances and covariances of the variables in a dataset.
  - By its definition, the covariance matrix is **positive semi-definite**.

University of Glasgow

Summary of Eigenproblems

- **Eigenvalues and eigenvectors are generally defined for square matrices A**.
- A matrix A transforms a general vector by rotating and scaling it. However, the eigenvectors of A are special because they **lie on a line that can only be scaled**, not rotated by the transform.
- The eigenvalues of A are the **scaling factors** λ_i that correspond to each eigenvector $\mathbf{x_i}$.
- Eigenvectors and eigenvalues **can be computed algorithmically** (e.g., by the power iteration algorithm for finding the leading eigenvector).
- **Eigendecomposition** is the factorisation of a square matrix A into three component matrices: $A = VDV^{-1}$ - it reveals the intrinsic structure and scaling of the matrix.
- The **eigenspectrum** is the set of all eigenvalues of the matrix. In PCA, we sort the absolute eigenvalues in descending order to determine component importance.
- If we have a **complete set of eigenvectors and eigenvalues, we can reconstruct the matrix**.
- **We can approximate a large matrix A with a few leading eigenvectors**; this is a simplified or truncated approximation to the original matrix.
- If we repeatedly apply a matrix A to an arbitrary vector x, the resulting vector will **eventually align with the direction of the principal (largest) eigenvector**.

Matrix Inversion

Four basic algebraic operations on matrices:

- scalar multiplication c$\mathbf{A}$;
- matrix addition $\mathbf{A+B}$;
- matrix multiplication $\mathbf{BA}$
- matrix transposition $\mathbf{A}^{\mathrm{T}}$

A further important operation: **inversion** $\mathbf{A}^{-1}$, defined such that $\mathbf{A}^{-1}\mathbf{A}=\boldsymbol{I}$:

- $\mathbf{A}^{-1}(\mathbf{Ax})=\mathbf{x}$,
- $(\mathbf{A}^{-1})^{-1}=\mathbf{A}$
- $(\mathbf{AB})^{-1}=\mathbf{B}^{-1}\mathbf{A}^{-1}$

Inversion is only defined for certain kinds of matrices: ***Square Matrices, Non-singular Matrices***

Computing the inverse of a matrix

- **Gauss-Jordan elimination method – row reduction – a standard algorithm**
  - not computationally efficient for large matrices.
- Instead, we often use the workhorse of matrix decompositions: the **singular value decomposition**, which we will discuss later.
- In the meantime, we can use the NumPy method *np.linalg.inv*, as shown below:

numpy.linalg.inv

`linalg.inv(a)` [source]

Compute the (multiplicative) inverse of a matrix.

Given a square matrix *a*, return the matrix *ainv* satisfying `dot(a, ainv) = dot(ainv, a) = eye(a.shape[0])`.

Parameters: a : *(..., M, M) array_like*
Matrix to be inverted.

Returns: ainv : *(..., M, M) ndarray or matrix*
(Multiplicative) inverse of the matrix *a*.

Raises: LinAlgError
If *a* is not square or inversion fails.

University of Glasgow

Inversion as "undo"

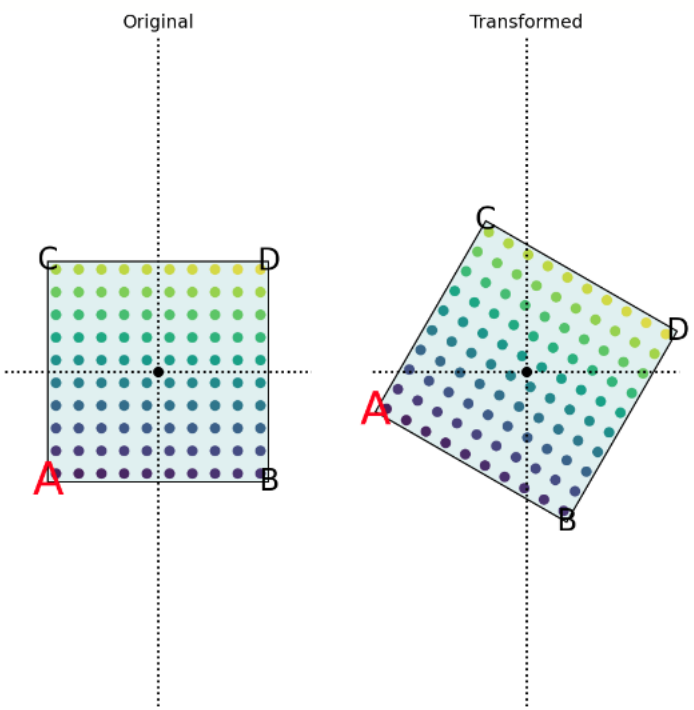
- Inversion of a matrix creates a new linear operator which reverses the original operation. Let's see it in action:

```
# Verify that left-multiplying by the inverse "undoes" the transformation applied by A to a ra
# Create a random vector
x = np.random.normal(0, 1, (3, 1))
print_matrix("x", x)
# Tranform it
print_matrix("Ax", A @ x)
# Left-multiply by the inverse to recover the original vector
print_matrix("A^{-1}(Ax)", np.linalg.inv(A) @ (A @ x))
```

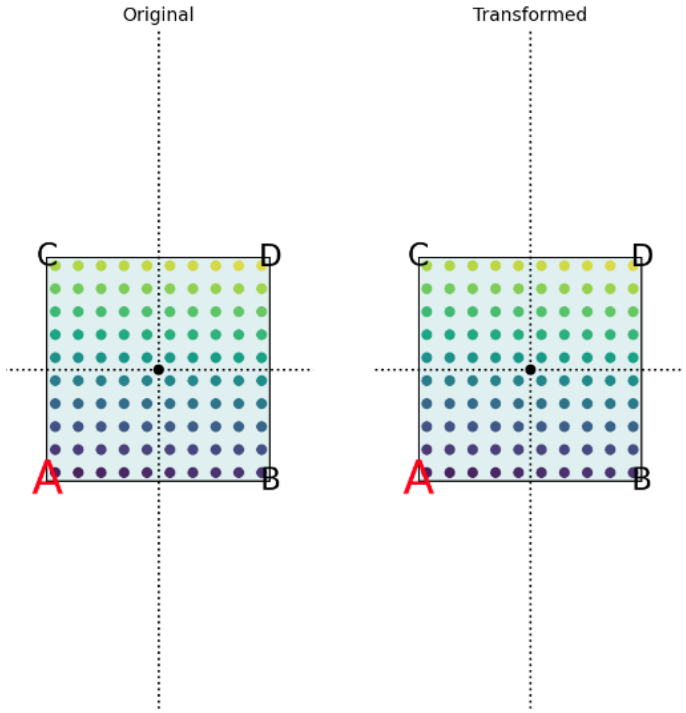
```
x
 [[-0.18]
 [ 0.95]
 [-0.92]]
Ax
 [[-0.28]
 [ 0.73]
 [-1.11]]
A^{-1}(Ax)
 [[-0.18]
 [ 0.95]
 [-0.92]]
```

Inversion as "undo"

- 30 degree rotation matrix



- 30 degree rotation matrix, left-multiplied by its inverse



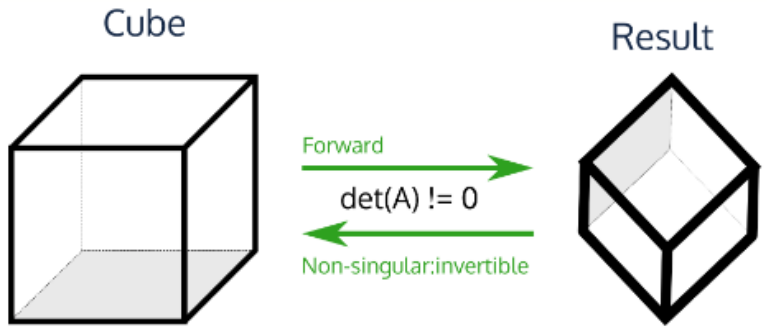
University of Glasgow

BUT only square matrices can be inverted !

- Inversion is only defined for square matrices, representing a linear transform $\mathbb{R}^n \rightarrow \mathbb{R}^n$.
- For the inverse to exist, the determinant of the matrix must be non-zero:
  - $\det(A) \neq 0$.
- Why?
  - Every output vector must come from one, and only one, input vector (a one-to-one mapping).
  - If det(A)=0 (a singular matrix), the transformation has collapsed at least one dimension.
  - This means that different input vectors map to the same output vector.
  - You can't reverse the process because you don't know which input vector to return to!

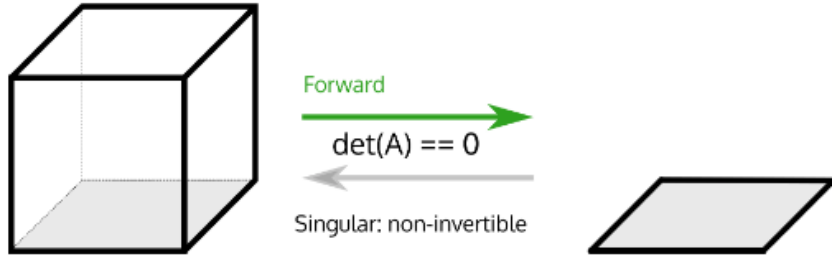
Singular and non-singular matrices

- A matrix with det(A)=0 is called **singular** and has no inverse.
- A matrix which is invertible is called **non-singular**.



Geometric intuition

- One of the dimensions of the parallelepiped has been squashed to nothing at all.
- It is impossible to reverse the transformation, because information was lost in the forward transform.



University of Glasgow

Revisiting the package distribution problems

Problem: "predicting" the past with inversion

- We can use **inversion** to solve the problem of predicting the distribution at $\mathbf{x}_{t=-1}$ given $\mathbf{x}_{t=0}$.

$$\mathbf{x}_{t=-1} = A^{-1}\mathbf{x}_{t=0}$$

- We can compute any negative power of the matrix to "undo" any number of steps:

$$A^{-k} = \underbrace{A^{-1}A^{-1}A^{-1}\dots A^{-1}}_{\text{k repetitions}}$$

Linear systems

- Imagine a system where there is an input, ***x***, and an output, ***y***.

$$f(x) = Ax = y$$

$$A = \begin{bmatrix} 0.5 & 1.0 & 2.0 \\ 1.0 & 0.5 & 0.0 \\ 0.6 & 1.1 & -0.3 \end{bmatrix}$$

$$\begin{aligned} y_1 &= 0.5x_1 + 1.0x_2 + 2.0x_3 \\ y_2 &= 1.0x_1 + 0.5x_2 + 0.0x_3 \\ y_3 &= 0.6x_1 + 1.1x_2 - 0.3x_3 \end{aligned}$$

This is a **linear system** or **linear system of equations**

Task: Given ***y,*** "predicting" the past **x**

Solving linear systems

- The solution of linear systems is apparently simple for cases where ***A*** is ***square***.
- If ***Ax=y***, then left-multiplying both sides by $\boldsymbol{A}^{-1}$ we get

$$A^{-1}A\mathbf{x} = A^{-1}\mathbf{y}$$
$$I\mathbf{x} = A^{-1}\mathbf{y}$$
$$\mathbf{x} = A^{-1}\mathbf{y}$$

- This only works for square matrices, as $\boldsymbol{A}^{-1}$ is not defined for non-square matrices. This means that $\mathbf{x}$ and $\mathbf{y}$ must have the same number of dimensions.

University of Glasgow

Solving Linear Systems (Row-Reduction)

- Reduce the *augmented matrix* into *reduced row-echelon form* (RREF)
- We accomplish this by elementary row operations:
  - Swap two rows
  - Multiply a row by a non-zero scalar
  - Add/subtract a multiple of one row to another row

- Steps:
  - Use row operations to create leading 1s – pivots
  - Initially create zeros below them, moving downwards
  - Create zeros above them, moving upwards

Solving Linear Systems (Row-Reduction) II

$$\begin{cases} x + y = 5, \\ 3x + 4y = 6. \end{cases}$$

$$\left[\begin{array}{cc|c} 1 & 1 & 5 \\ 3 & 4 & 6 \end{array}\right] \xrightarrow{R_2 \leftarrow R_2 - 3R_1} \left[\begin{array}{cc|c} 1 & 1 & 5 \\ 0 & 1 & -9 \end{array}\right] \xrightarrow{R_1 \leftarrow R_1 - R_2} \left[\begin{array}{cc|c} 1 & 0 & 14 \\ 0 & 1 & -9 \end{array}\right]$$

Row Echelon Form (REF)

Reduced Row Echelon Form (RREF)

$$x = 14, y = -9$$

We can stop here and solve by back substitution.

(Gaussian elimination)

This corresponds to the solved system

(Gauss-Jordan elimination)

University of Glasgow

How to Find the Rank of a Matrix

- Rank is effectively the number of pivots (leading 1s) in the REF of a matrix; equivalently, **the number of non-zero rows** in the REF.
  - (No need to reach a RREF.)

$$A = \begin{bmatrix} 1 & 2 & 3 \\ 4 & 5 & 6 \\ 7 & 8 & 9 \end{bmatrix} \longrightarrow \mathrm{REF}(A) = \begin{bmatrix} 1 & 2 & 3 \\ 0 & 1 & 2 \\ 0 & 0 & 0 \end{bmatrix} \Rightarrow \quad \mathrm{rank}(A) = 2.$$

- We don't need to use the augmented matrix here – equivalent to centering the lines (or planes)
- A square matrix $A \in \mathbb{R}^{n \times n}$ is non-singular (invertible) if rank(A) = n
- If A is singular (non-invertible), i.e. if rank(A) < n, it follows that det(A) = 0

Singularity from a Geometric Perspective

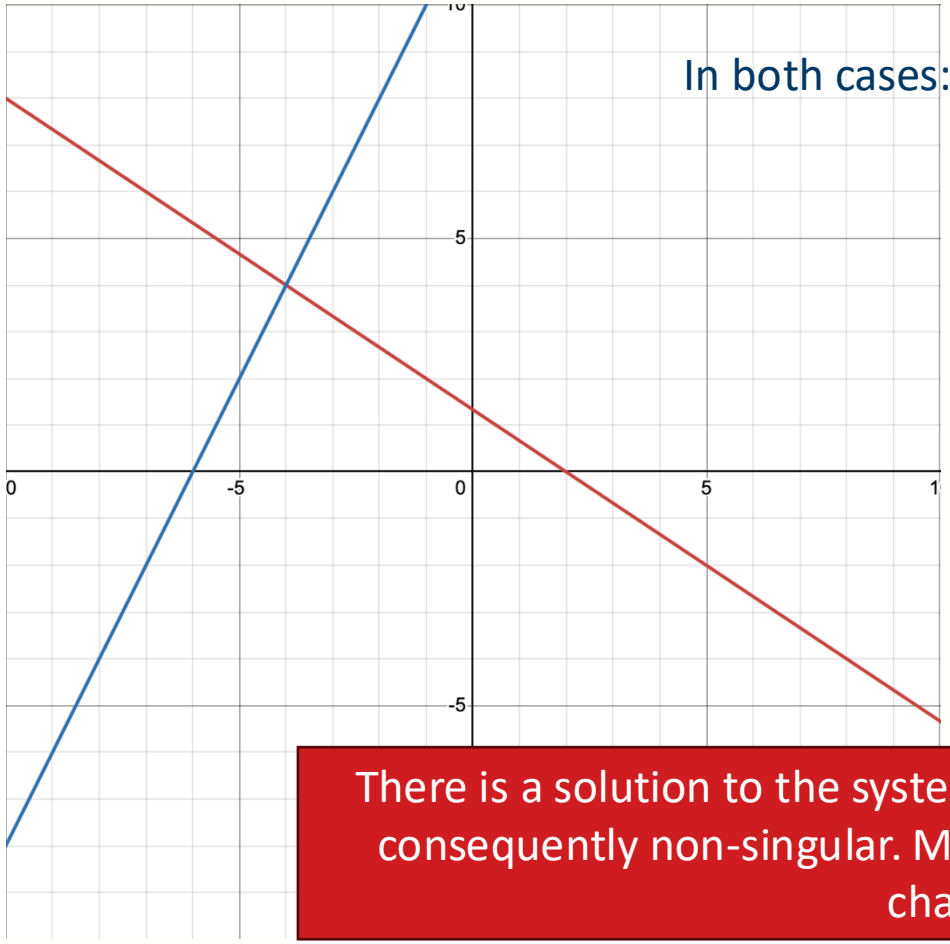
2x+3y=4
-2x+y=12

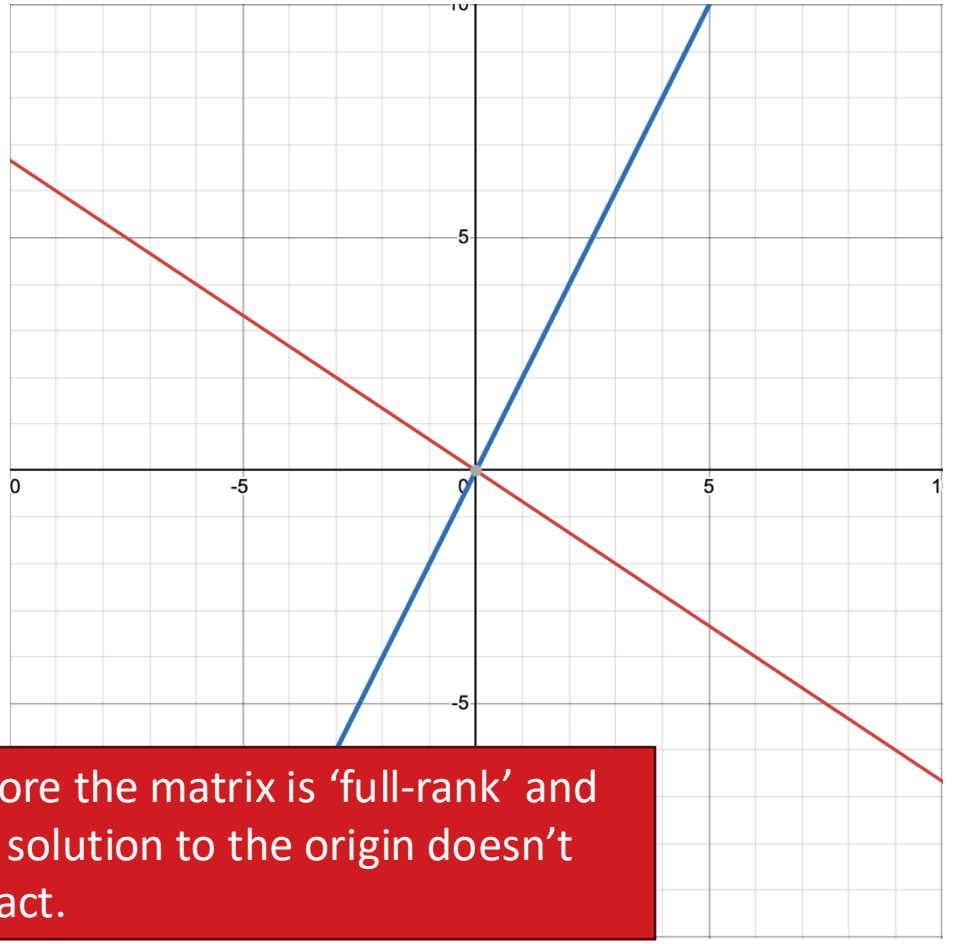
$$\left[\begin{array}{cc|c} 2 & 3 & 4 \\ -2 & 1 & 12 \end{array}\right]$$

2x+3y=0
-2x+y=0

$$\left[\begin{array}{cc|c} 2 & 3 & 0 \\ -2 & 1 & 0 \end{array}\right]$$

In both cases: $\begin{bmatrix} 2 & 3 \\ 0 & 1 \end{bmatrix}$





There is a solution to the system. Therefore the matrix is ‘full-rank’ and consequently non-singular. Moving the solution to the origin doesn’t change this fact.

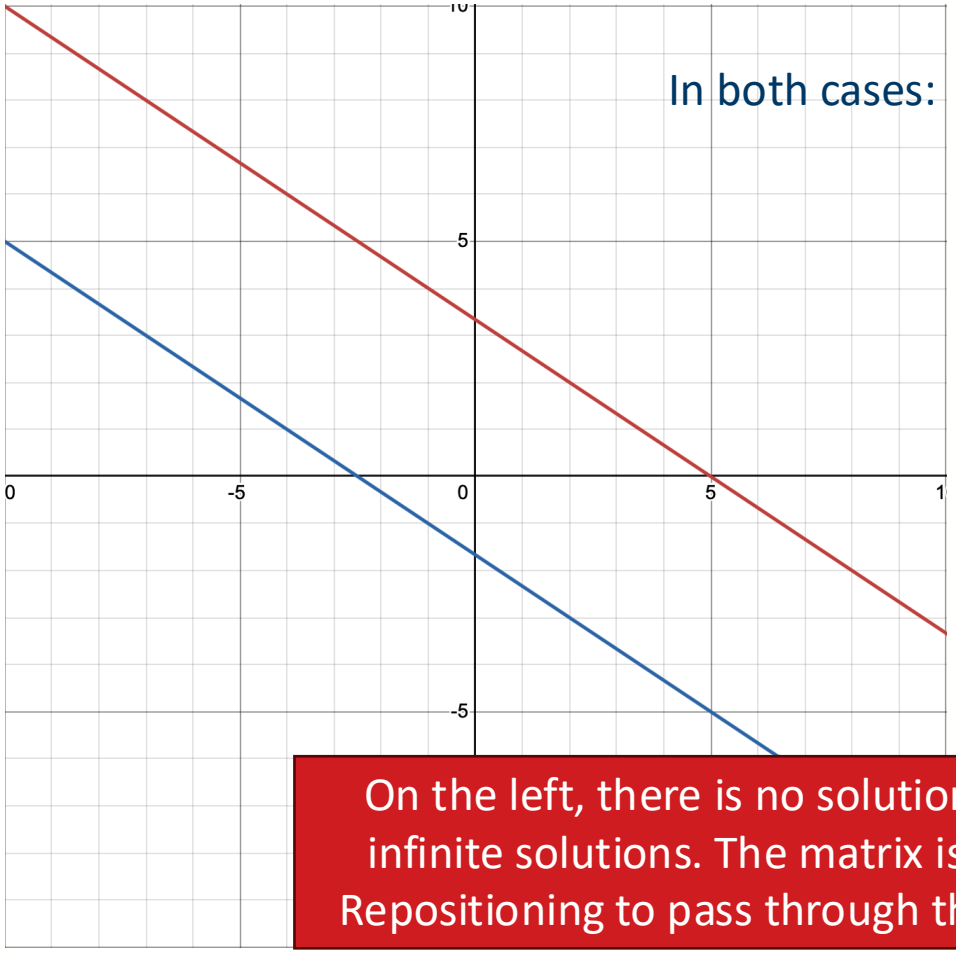
Singularity from a Geometric Perspective II

2x+3y=10
-4x-6y=10

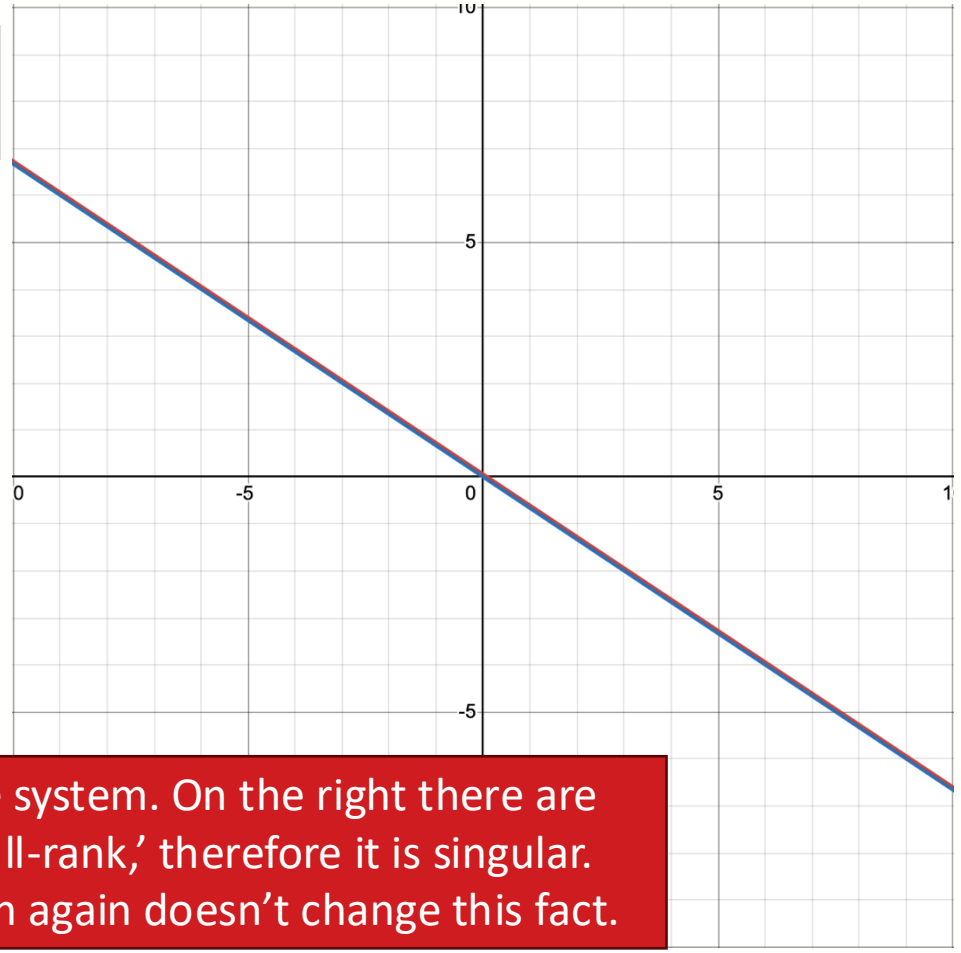
$$\left[\begin{array}{cc|c} 2 & 3 & 10 \\ -4 & -6 & 10 \end{array}\right]$$

2x+3y=0
-4x-6y=0

$$\left[\begin{array}{cc|c} 2 & 3 & 0 \\ -4 & -6 & 0 \end{array}\right]$$



In both cases: $\begin{bmatrix} 2 & 3 \\ 0 & 0 \end{bmatrix}$



On the left, there is no solution to the system. On the right there are infinite solutions. The matrix is not 'full-rank,' therefore it is singular. Repositioning to pass through the origin again doesn't change this fact.

University of Glasgow

Approximate solutions for linear systems

- In practice, linear systems are almost never solved with a direct inversion.
  - Inversion is $O(N^3)$
  - ***Nonsquare*** *in nature*

- Instead, linear systems are typically solved iteratively, either using specialised algorithms based on knowledge of the structure of the system, or using **optimisation**, which will be the topic of the next Unit (lecture 4).
  - E.g., to find solutions that minimise $\|A\mathbf{x}-\mathbf{y}\|_2$ by adjusting the value of $\mathbf{x}$ repeatedly.

Singular Value Decomposition (SVD)

- Eigendecompositions only apply to **diagonalisable** matrices; which are a subset of **square matrices**.
- There are many problems which have **non-square** matrices which we would like to be able to decompose.
- The **singular value decomposition (SVD)** is a general approach to decomposing ANY matrix $\boldsymbol{A}$. It is the powerhouse of computational linear algebra.
- The SVD produces a decomposition which splits ***ANY*** matrix up into three matrices:

$$\boldsymbol{A=U\Sigma V^T}$$

Singular value decomposition

- The SVD produces a decomposition which splits ***ANY*** matrix up into three matrices:

$$A = U\Sigma V^T$$

where

- A is any m×n matrix,
- U is a **square orthogonal** m×m matrix, whose columns contain the **left singular vectors**,
- V is an **square orthogonal** n×n matrix, whose columns contain the **right singular vectors**,
- Σ is a diagonal m×n matrix, whose diagonal contains the **singular values**.

Orthogonal (real-valued) matrices: $U^T = U^{-1}$ i.e., whose rows all have unit norm and whose columns also all have unit norm.

University of Glasgow

Eigendecomposition vs SVD

- **Eigendecomposition**: Can only be applied to square matrices.
- **SVD**: Can be applied to **ANY** matrix (square or rectangular).
- If $\boldsymbol{A}$ is a symmetric positive semi-definite matrix (**like a covariance matrix**), then the eigendecomposition and SVD give related results.

The SVD is the same as:

- taking the eigenvectors of $\boldsymbol{A^TA}$ to get $\boldsymbol{U}$
  - NB: $\boldsymbol{A^TA}$ is a symmetric matrix
- taking the square root of the *absolute* value of the eigenvalues $\boldsymbol{\lambda_i}$ of $\boldsymbol{A^TA}$ to get $\boldsymbol{\Sigma_i} = \sqrt{|\boldsymbol{\lambda_i}|}$
- taking the eigenvectors of $\boldsymbol{AA^T}$ to get $\boldsymbol{V^T}$

University of Glasgow

SVD decomposes any matrix into three matrices with special forms

- The SVD produces a decomposition which splits ***ANY*** matrix up into three matrices:

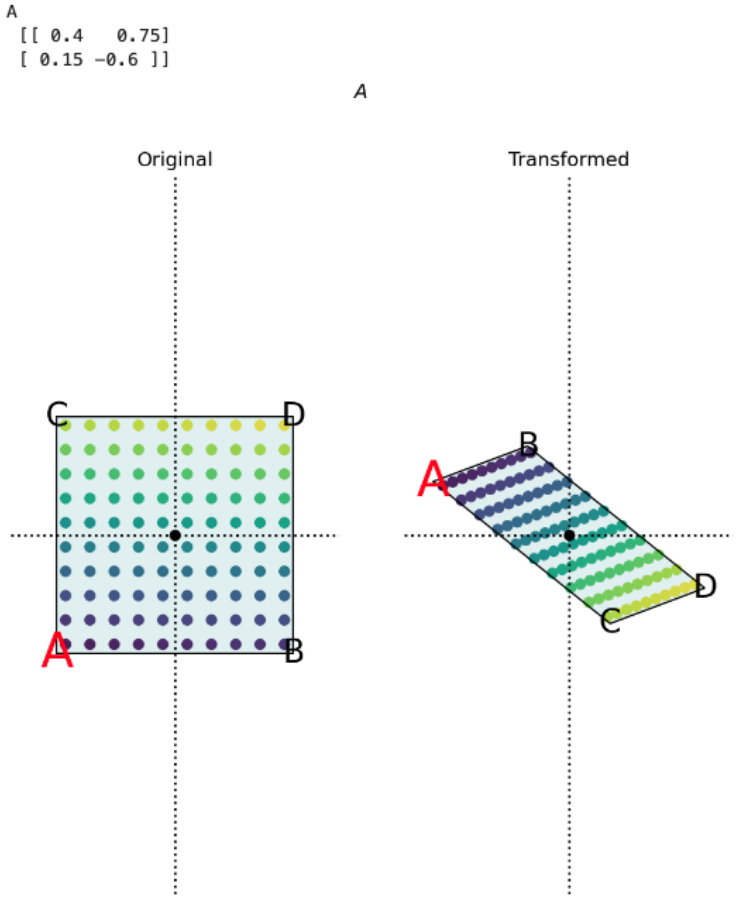
$$A=U\Sigma V^T$$

Special forms of matrices, like orthogonal matrices and diagonal matrices, are much easier to work with than general matrices. This is the power of the SVD.

- **U** is orthogonal, so is a pure rotation matrix,
- **Σ** is diagonal, so is a pure scaling matrix,
- **V** is orthogonal, so is a pure rotation matrix.

Rotate, scale, rotate

- *The SVD splits any matrix transformation into a rotate-scale-rotate operation.*




```
: u, sigma, vt = np.linalg.svd(A)

show_matrix_effect(vt, "V^T")
show_matrix_effect(np.diag(sigma) @ vt, "\Sigma V^T")
show_matrix_effect(u @ np.diag(sigma) @ vt, "U \Sigma V^T")

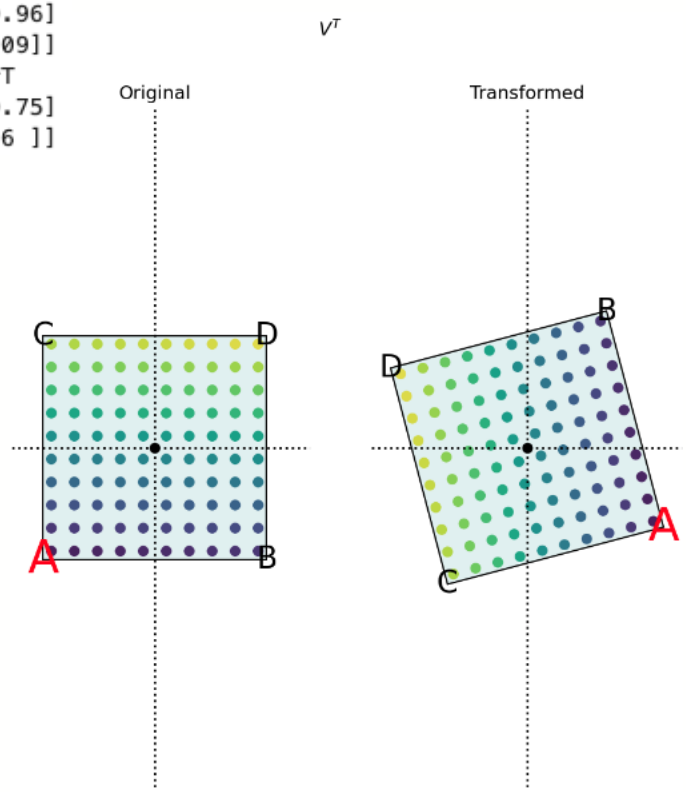
#Sanity
print("Determinant of vt:" ,np.linalg.det(vt))
print("Determinant of u:" ,np.linalg.det(u))
print("Determinant of Sigma:" ,np.linalg.det(np.diag(sigma)))
```

```
V^T
 [[-0.26 -0.97]
 [ 0.97 -0.26]]
\Sigma V^T
 [[-0.25 -0.96]
 [ 0.34 -0.09]]
U \Sigma V^T
 [[ 0.4   0.75]
 [ 0.15 -0.6 ]]
Determinant of vt: 1.0
Determinant of u: -1.0
Determinant of Sigma: 0.3525
```

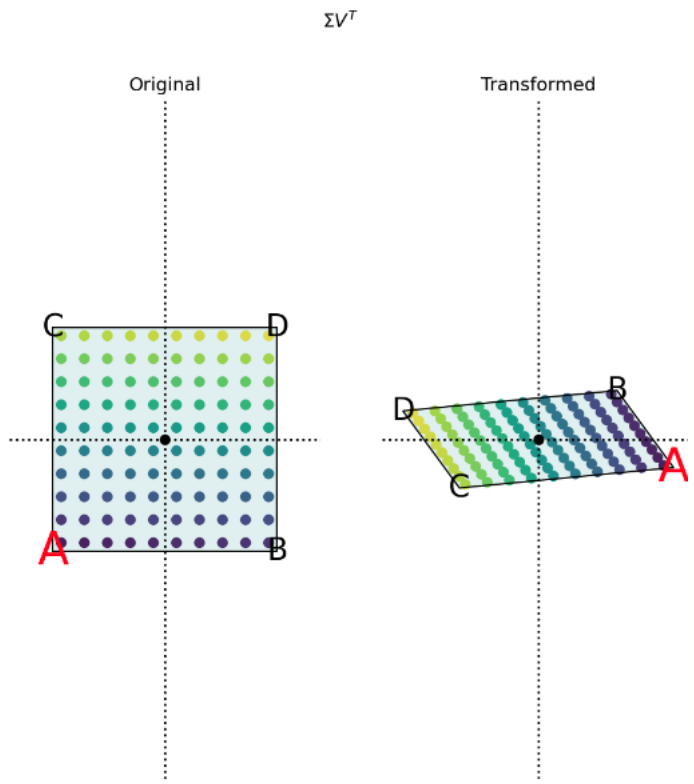
Rotate, scale, rotate

```
V^T
 [[-0.26 -0.97]
 [ 0.97 -0.26]]
\Sigma V^T
 [[-0.25 -0.96]
 [ 0.34 -0.09]]
U \Sigma V^T
 [[ 0.4   0.75]
 [ 0.15 -0.6 ]]
```

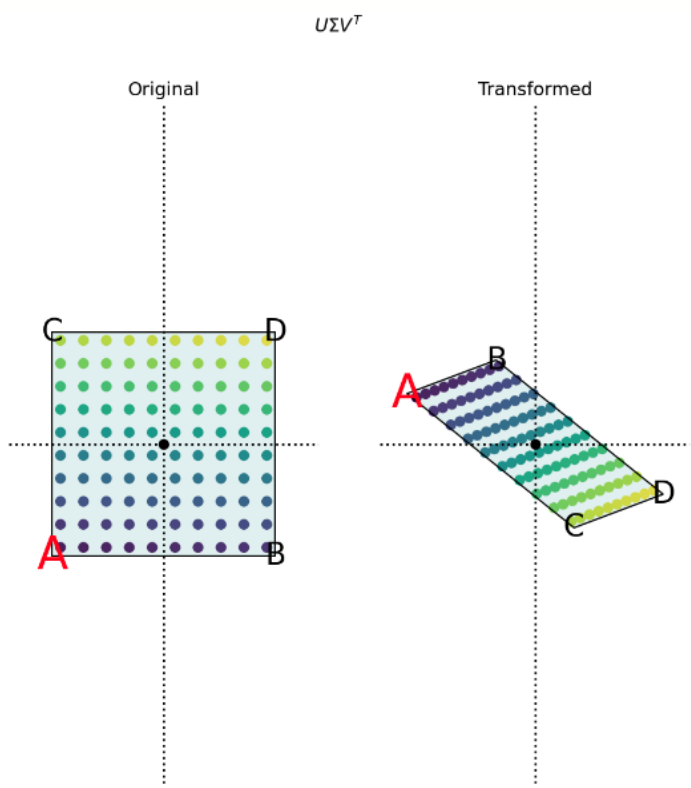
Rotate



Scale



Rotate



University of Glasgow

Rotate, scale, rotate

A=UΣV^T: U is orthogonal, so is a pure rotation matrix, ***Σ*** is diagonal, so is a pure scaling matrix, ***V*** is orthogonal, so is a pure rotation matrix.

Why does an orthogonal matrix only do pure rotation?

- **Orthogonal Matrix：** $U^{-1} = U^T$, $V^{-1} = V^T$
- **Preservation of Length**
  - $\|x\|_2 = x^T x;\ U^T U = U^{-1} U = I$
  - $\|Ux\|_2 = (Ux)^T (Ux) = x^T U^T U x = x^T x = \|x\|_2$
  - Therefore, the length of x is preserved after the transformation.
- **Preservation of Angles**:
  - $x \cdot y = x^T y$
  - $(Ux) \bullet (Uy) = (Ux)^T (Uy) = x^T U^T U y = x^T y$
  - Since the dot product remains unchanged, the angle between the vectors is preserved.

University of Glasgow

Using the SVD

- **Fractional powers:** We can use the SVD to compute interesting matrix functions like the square root of a matrix $\mathbf{A}^{1/2}$.
- **Invert a matrix:** $\mathbf{A}^{-1}$, which will "undo" the operation.

- The rule is simple: to do any of these operations, ignore U and V (which are just rotations), and apply the function to the singular values elementwise:

$$A^n = V\Sigma^n U^T$$

University of Glasgow

Inversion - relation to SVD

Invert a matrix: A^{-1}, which will "undo" the operation.

$$A^{-1} = V\Sigma^{-1}U^T$$

- This can be computed in O(n) time because $\boldsymbol{\Sigma}^{-1}$ can be computed simply by taking the reciprocal of each of the diagonal elements of $\boldsymbol{\Sigma}$.

- *N.B. No free lunch! As a consequence, we now know that computing the SVD must take* $O(n^3)$ *time for square matrices, since inversion cannot be achieved faster than* $O(n^3)$.

University of Glasgow

Pseudo-inverse (non-symmetric)

- We can also pseudo-invert a matrix: $A+$, which will approximately "undo" the operation, even when A isn't square.

- The pseudo-inverse of A is just

$$A^{+} = V\Sigma^{-1}U^{T}$$

which is the same as the standard inverse computed via SVD, but taking care that Σ is the right shape - appropriate zero padding is required!

Fortunately, this is taken care of by the Numpy method ***np.linalg.pinv***.

University of Glasgow

A few more matrix properties

Rank of a matrix

- The **rank** of a matrix is equal to the number of non-zero singular values.
- If the number of non-zero singular values is equal to the size of the matrix, then the matrix is **full rank**.
- A full rank matrix has a non-zero determinant and will be invertible.
- The rank tells us how many dimensions the parallelotope that the transform represents will have.
- If a matrix does not have full rank, it is **singular** (non-invertible) and has **deficient rank**.
- If the number of non-zero singular values is much less than the size of the matrix, the matrix is **low rank**.

University of Glasgow

A few more matrix properties

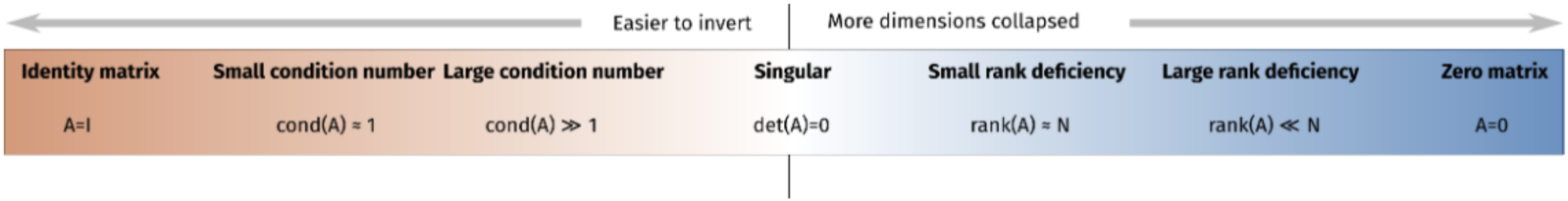
Condition number of a matrix

- The **condition number** number of a matrix is the ratio of the largest singular value to the smallest.
- This is only defined for full rank matrices.
- The condition number measures how sensitive inversion of the matrix is to small changes.
- A matrix with a small condition number is called **well-conditioned** and is unlikely to cause numerical issues.
- A matrix with a large condition number is **ill-conditioned**, and numerical issues are likely to be significant.
- An ill-conditioned matrix is almost singular, so inverting it will lead to invalid results due to floating point roundoff errors.

University of Glasgow

Relation to singularity

- A **singular** matrix A is un-invertible and has det(A)=0. Singularity is a binary property and is either true or false.
- We can think of **rank** as measuring "how singular" the matrix is, i.e. how many dimensions are lost in the transform.
- We can think of the **condition number** as measuring how close a non-singular matrix is to being singular. A matrix which is nearly singular may become effectively singular due to floating point roundoff errors.



University of Glasgow

Applying decompositions

- Whitening a data set (self-study in the lab)
  - **Whitening** removes all linear correlations within a dataset.
  - It is a normalisation step used to standardise data before analysis.
- Whitening does the following:
  - centers the data around its mean, so it has **zero mean**.
  - "squashes" the data so that its distribution is **spherical** and has **unit covariance**.
    - '**spherical**' means that it is roughly spread out evenly in all directions in the space it's plotted in.
    - '**unit covariance**' means the diagonal elements are all 1s. This means each feature (or dimension) has a variance of 1.
      - We can normalise to 1 by: 1.0 / np.sqrt(sigma)

Whitening a data set

1. **Center the Data:** Before applying SVD, you should first center the data by subtracting the mean of each feature.

$$X_{\text{centered}} = X - \text{mean}(X)$$

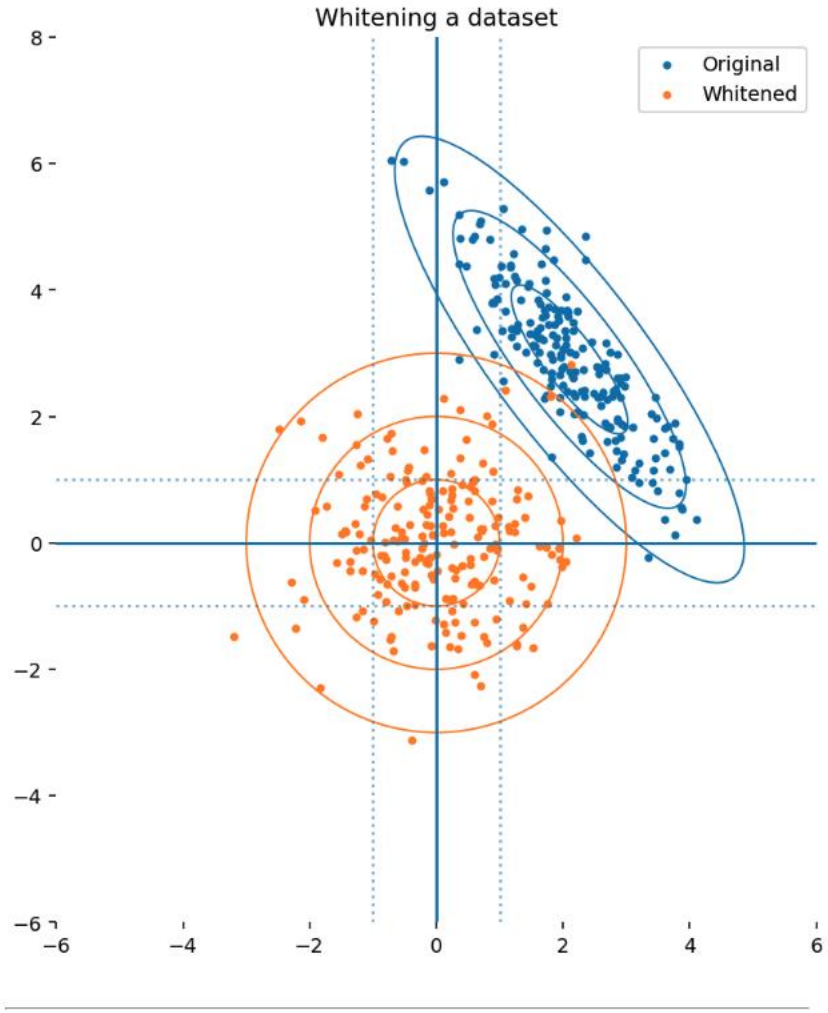
2. **Apply SVD:** For the centered data X_{centered}, perform SVD:

$$X_{\text{centered}} = U\Sigma V^T$$

3. **Whitening Transform:** The whitening transform can be computed using U and Σ. Specifically, the whitened data X_{whitened} is given by:

$$X_{\text{whitened}} = U\Sigma^{-1/2}U^T X_{\text{centered}}$$

Here, $\Sigma^{-1/2}$ is a diagonal matrix where each element is the **inverse square root** of the corresponding **singular value** in Σ.
This makes all variances = 1



University of Glasgow

PCA vs Whitening a data set

- Whitening can be seen as an extension of the PCA process.
  - Both SVD (more numerically stable) and eigendecomposition can be used in both the tasks
  - But their objectives are different
- Objectives
  - **PCA**: Reduce the dimensionality of data while retaining as much variance as possible.
    - Only keep *k* most significant components.
    - Speed up your unsupervised feature learning algorithm
  - **Whitening**: Transform the data such that it has an identity covariance matrix.
    - No need to find *k* most significant components.
    - An important pre-processing step for many algorithms



Thank you

Contact:

iraklis.klampanos@glasgow.ac.uk